

University of Rajshahi · Dept. of Computer Science & Engineering

CSE4121

Object Oriented Design and Design Patterns

সম্পূর্ণ থিওরি — বাংলায় বিস্তারিত ব্যাখ্যা

Part A: Design Principles · Part B: Design Patterns · UML · বাংলা কमेंটসহ Java কোড

এই বইয়ে যা আছে

Part-A slides-এর সব টপিক (OOAD, CRC, চারটি design principle, coupling-cohesion, SoC, conceptual integrity, code smells, SOLID, sequence ও state diagram, model checking) এবং Section-B notes-এর ১১টি pattern, সাথে পরীক্ষায় আসা Facade, Chain of Responsibility ও MVC। হেডিং ইংরেজিতে, ব্যাখ্যা বাংলায়। প্রতিটি টপিকের শেষে PDF 3 (English Q&A)-এর প্রশ্ন নম্বর দেওয়া আছে।

1	How to Use This Book	1
1.1	Exam Pattern	1
2	Object-Oriented Analysis and Design	2
2.1	What is Software Design and Why Does It Matter	2
2.2	Requirements: Functional and Non-Functional	2
2.3	Conceptual Design vs Technical Design	2
2.4	Quality Attributes and Design Trade-offs	3
2.5	CRC Cards	3
2.6	Object-Oriented Modelling and UML	4
2.7	Entity, Boundary and Control Objects	4
2.8	Evolution of Programming Languages	5
2.9	Software Design vs Software Architecture	5
2.10	Benefits of Modelling Real-World Entities as Objects	5
3	The Four Object-Oriented Design Principles	6
3.1	Abstraction	6
3.2	Encapsulation	7
3.3	Decomposition	8
3.4	Generalization and Inheritance	10
3.5	UML Class Diagram Notation Cheat-Sheet	13
4	Design Complexity and Code Quality	14
4.1	Coupling	14
4.2	Cohesion	14
4.3	The Control-Flag Problem: FarmController	14
4.4	Separation of Concerns	15
4.5	Information Hiding	16
4.6	Conceptual Integrity	17
4.7	Code Smells	17
4.8	SOLID Principles and the Law of Demeter	18
5	Behavioural Modelling with UML	19
5.1	UML Sequence Diagram	19
5.2	UML State Diagram	20
5.3	Model Checking	21
6	Design Pattern Foundations	22
6.1	What is a Design Pattern	22

6.2	Why Learn Design Patterns	22
6.3	Two GoF Principles of Reusable Design	22
6.4	Categories of Design Patterns	23
7	Creational Patterns	24
7.1	Singleton Pattern	24
7.2	Factory Pattern	25
7.3	Abstract Factory Pattern	27
7.4	Builder Pattern	28
8	Structural Patterns	31
8.1	Adapter Pattern	31
8.2	Bridge Pattern	33
8.3	Filter (Criteria) Pattern	34
8.4	Composite Pattern	35
8.5	Facade Pattern	36
8.6	Decorator Pattern (Extra)	37
9	Behavioural Patterns and MVC	38
9.1	Command Pattern	38
9.2	Mediator Pattern	39
9.3	Observer Pattern	40
9.4	Chain of Responsibility Pattern	42
9.5	MVC (Model–View–Controller)	44
10	Quick Revision Tables	46
10.1	All Patterns at a Glance	46
10.2	Scenario Keywords → Pattern	46
10.3	Section A One-Page Summary	47

অধ্যায় ১

How to Use This Book

এই বইটি CSE4121 (Object Oriented Design and Design Patterns) কোর্সের পুরো সিলেবাসের থিওরি বাংলায় ব্যাখ্যা করে। হেডিং ও টেকনিক্যাল শব্দগুলো ইংরেজিতে রাখা হয়েছে, কারণ পরীক্ষায় উত্তর ইংরেজিতেই লিখতে হবে। প্রতিটি টপিকের সাথে UML ডায়াগ্রাম এবং বাংলা কमेंটসহ Java কোড দেওয়া আছে।

তিনটি PDF একসাথে যেভাবে ব্যবহার করবে

PDF 1 (Prediction): কোন টপিক আসার সম্ভাবনা কত, আর দুই সেট মডেল প্রশ্ন।

PDF 2 (এই বই): প্রতিটি টপিক গভীরভাবে বোঝা।

PDF 3 (English Q&A): পরীক্ষার খাতায় যেভাবে লিখবে ঠিক সেভাবে ইংরেজি উত্তর। এই বইয়ে “Q&A: Q21” লেখা থাকলে বুঝবে PDF 3-এর প্রশ্ন ২১-এ এর পূর্ণ উত্তর আছে।

1.1 Exam Pattern

পরীক্ষা ২ ঘণ্টার, পূর্ণমান ৩৫। Section A (প্রশ্ন ১-৩) ও Section B (প্রশ্ন ৪-৬) থেকে দুটি করে মোট চারটি প্রশ্নের উত্তর দিতে হয়; প্রতিটি প্রশ্ন ৮.৭৫ নম্বরের। Section A-তে থাকে ডিজাইন প্রিন্সিপল (Part A slides), Section B-তে ডিজাইন প্যাটার্ন (Section B notes)। ২০২৪ সাল থেকে এবং দুটি ক্লাস টেস্টেই প্রশ্নগুলো **Dhaka Urban Management System (DUMS)** আর **Agricultural Automation System (AAS)** সিনারিও দিয়ে করা হচ্ছে, তাই এই বইয়ের উদাহরণগুলোও মূলত এই দুই সিস্টেম থেকে নেওয়া।

সবচেয়ে বেশি আসা টপিক (প্রতি বছর)

Association/Aggregation/Composition + UML, CRC card, Design vs Architecture, Singleton, Adapter, “What are design patterns”। এরপর Observer, Chain of Responsibility, coupling-cohesion, inheritance-এর ভুল ব্যবহার, conceptual integrity, sequence diagram।

Object-Oriented Analysis and Design

2.1 What is Software Design and Why Does It Matter

সফটওয়্যার তৈরির প্রক্রিয়াকে সাধারণত কয়েকটি ধাপে ভাগ করা হয়: **Requirements** → **Design** → **Implementation** → **Testing** → **Deployment** → **Maintenance**। Requirements বলে সিস্টেমকে কী করতে হবে, আর কোড বলে কম্পিউটার কীভাবে তা করবে। এই দুইয়ের মাঝখানের সেতু হলো **design**।

Object-Oriented Analysis and Design (OOAD) হলো এমন একটি পদ্ধতি যেখানে সমস্যাটিকে বাস্তব জগতের **object**-এর মাধ্যমে বিশ্লেষণ (analysis) করা হয় এবং তারপর সেই object-গুলোর দায়িত্ব, সম্পর্ক ও পারস্পরিক যোগাযোগ ঠিক করে সমাধানের নকশা (design) তৈরি করা হয়।

কোড লেখার আগে ডিজাইন কেন

- **ভুল সংশোধন সস্তা:** ডায়াগ্রামে একটা বক্স সরাতে কয়েক মিনিট লাগে; কিন্তু যে ক্লাস আরও বিশটা ক্লাস ব্যবহার করছে সেটা বদলাতে কয়েকদিন লাগে।
- **সঠিক কাঠামো আগে ঠিক হয়:** কোন object থাকবে, কার কী দায়িত্ব — এগুলো ঠিক থাকলে কোডিং শুধু অনুবাদের কাজ হয়ে যায়।
- **টিমওয়ার্ক:** interface আগে ঠিক থাকলে কয়েকজন ডেভেলপার আলাদা আলাদা মডিউল একসাথে বানাতে পারে।
- **Quality আগে থেকেই পরিকল্পিত:** performance, security, maintainability ইত্যাদি বিষয় বিকল্পগুলো খোলা থাকতেই বিবেচনা করা যায়।

2.2 Requirements: Functional and Non-Functional

ডিজাইন শুরু হয় requirements বোঝা দিয়ে। Client অনেক সময় নিজের চাহিদা পরিষ্কারভাবে বলতে পারে না, তাই প্রশ্ন করে, উদাহরণ দেখিয়ে, prototype বানিয়ে আসল প্রয়োজনটা বের করতে হয় (requirements elicitation)।

- **Functional requirement:** সিস্টেম কী কাজ করবে। যেমন DUMS-এ “নাগরিক ছবি ও লোকেশনসহ ময়লা না তোলার অভিযোগ জমা দিতে পারবে”।
- **Non-functional requirement:** কাজটা কতটা ভালোভাবে বা কোন সীমাবদ্ধতার মধ্যে করবে। যেমন “১০,০০০ জন একসাথে ব্যবহার করলেও অভিযোগের পেজ ২ সেকেন্ডের মধ্যে লোড হবে”, “ব্যক্তিগত তথ্য encrypted থাকবে”।

2.3 Conceptual Design vs Technical Design

Conceptual design হলো উচ্চ-স্তরের নকশা: সিস্টেমে কী কী প্রধান component থাকবে এবং প্রত্যেকটার দায়িত্ব কী। এখানে কোনো প্রযুক্তি (ভাষা, ডেটাবেস) ঠিক করা হয় না। এর artefact হলো conceptual mock-up, component-এর তালিকা, CRC card।

Technical design হলো নিম্ন-স্তরের নকশা: প্রতিটি component কীভাবে তৈরি হবে — কোন ক্লাস, কোন মেথড, কোন ডেটা স্ট্রাকচার, কোন অ্যালগরিদম, component-গুলো কীভাবে যোগাযোগ করবে। এর artefact হলো UML class/sequence/state diagram, API, database schema। Technical design প্রতিটি conceptual component-কে ভাঙতে থাকে যতক্ষণ না প্রতিটি অংশ সরাসরি কোড করার মতো ছোট হয়।

	Conceptual design	Technical design
প্রশ্ন	কী কী component লাগবে, কে কী করবে?	প্রতিটা কীভাবে বানানো হবে?
স্তর	High-level, প্রযুক্তি-নিরপেক্ষ	Low-level, প্রযুক্তি-নির্ভর
কার জন্য	Client, analyst, developer	Developer, tester
DUMS উদাহরণ	Smart bin, collection zone, truck, scheduler, citizen app; “bin নিজের ভরাট অবস্থা জানাবে”	GarbageBin, FillLevelSensor ক্লাস; প্রতি ১০ মিনিটে reading; shortest-path দিয়ে নিকটতম truck

2.4 Quality Attributes and Design Trade-offs

একটি ভালো ডিজাইন শুধু সঠিক কাজ করলেই হয় না, কিছু গুণগত (quality attributes) থাকতে হয়:

Attribute	অর্থ
Performance	কত দ্রুত সাজা দেয়, কত কম resource লাগে
Security	অননুমোদিত ব্যবহার ও ডেটা চুরি থেকে সুরক্ষা
Maintainability	ভুল ঠিক করা ও পরিবর্তন করা কতটা সহজ
Usability	ব্যবহারকারীর জন্য কতটা সহজ
Scalability	ব্যবহারকারী বাড়লেও ঠিকমতো চলে কিনা
Reusability	অংশগুলো অন্য সিস্টেমে আবার ব্যবহার করা যায় কিনা
Availability	কতক্ষণ সিস্টেম চালু ও ব্যবহারযোগ্য থাকে

এই গুণগুলো প্রায়ই একে অপরের বিপরীতে কাজ করে, তাই **trade-off** করতে হয়। যেমন বেশি security (বারবার OTP) দিলে usability কমে; কোড খুব optimize করলে performance বাড়ে কিন্তু maintainability কমে। আবার **client-এর চাহিদা** (বেশি feature, দ্রুত ডেলিভারি, কম খরচ) আর **developer-এর scope** (সময়, বাজেট, দক্ষতা, সার্ভার ক্ষমতা)-এর মধ্যেও ভারসাম্য রাখতে হয়। সমাধানের উপায়: requirement-গুলোর অগ্রাধিকার ঠিক করা, prototype দেখানো, মাপা যায় এমন quality target ঠিক করা (“অবস্থান সর্বোচ্চ ৬০ সেকেন্ড পুরনো”), ধাপে ধাপে release। (Q&A: Q8)

2.5 CRC Cards

CRC (Class–Responsibility–Collaborator) card হলো conceptual design-এর একটি সহজ কৌশল। একটি ছোট কার্ডে তিনটি অংশ থাকে: উপরে **Class name**, বাঁ দিকে **Responsibilities** (ক্লাসটি কী জানে ও কী করে), ডান দিকে **Collaborators** (দায়িত্ব পালনে কোন কোন ক্লাসের সাহায্য লাগে)।

CRC card তৈরির ধাপ

1. সিনারিও পড়ে **বিশেষ্য (noun)** দাগাও — এগুলো সম্ভাব্য ক্লাস।
2. **ক্রিয়া (verb)** দাগাও — এগুলো সম্ভাব্য responsibility।
3. প্রতিটি ক্লাসের জন্য একটি কার্ড লেখো। “জানে” ধরনের responsibility পরে attribute হবে, “করে” ধরনের responsibility মেথড হবে, collaborator পরে association হবে।
4. টেবিলে কার্ডগুলো রেখে সিনারিওটা **role-play** করো: কোন কার্ড কাকে অনুরোধ পাঠায়। কোনো ধাপ কোনো কার্ড করতে না পারলে বুঝবে একটা ক্লাস বা দায়িত্ব বাদ পড়েছে।
5. যে কার্ডে লেখা উপচে পড়ছে তাকে ভাগ করো (cohesion বাড়াতে), যে কার্ড প্রায় কিছুই করে না তাকে অন্যটার সাথে মিলিয়ে দাও।

উদাহরণ: DUMS login ও সেবা নির্বাচন

Class	Responsibilities	Collaborators
Citizen	NID, নাম, ফোন জানে; login করে; অভিযোগ দেয়; বিল দেয়	SmartCityPortal, Complaint, Payment
SmartCityPortal	ID-password নেয়; সেবার মেনু দেখায়; ফলাফল দেখায়	AuthenticationService, Complaint, Payment
AuthenticationService	ID-password যাচাই করে; session তৈরি করে	MunicipalDatabase
Complaint	ধরন, লোকেশন, অবস্থা জানে; নিজের অবস্থা হালনাগাদ করে	MunicipalService, MunicipalDatabase
Payment	বিল নম্বর ও পরিমাণ জানে; পেমেন্ট প্রসেস করে; রসিদ দেয়	Citizen, PaymentGateway

CRC card-এর সুবিধা

খরচ নেই, কাগজেই হয়; ক্লাসের ডেটা নয় বরং দায়িত্ব নিয়ে ভাবতে শেখায়; ছোট কার্ড হওয়ায় ক্লাস cohesive থাকে; role-play করে বাদ পড়া ক্লাস ধরা পড়ে; client সহ পুরো দল আলোচনায় অংশ নিতে পারে; কার্ড থেকে সরাসরি UML class diagram বানানো যায়। (Q&A: Q4, Q5)

2.6 Object-Oriented Modelling and UML

Object-oriented modelling হলো সমস্যা ও তার সমাধানকে পরস্পর যোগাযোগকারী কিছু object হিসেবে উপস্থাপন করা — কোড লেখার আগেই মডেলের মাধ্যমে। প্রথমে বাস্তব জগতের একটি conceptual model তৈরি হয়, পরে সেটাকে technical model-এ রূপান্তর করা হয় যা সরাসরি ক্লাসে পরিণত হয়।

UML (Unified Modeling Language) হলো সফটওয়্যার মডেল আঁকার একটি আদর্শ দৃশ্যমান ভাষা। প্রধান ডায়াগ্রামগুলো:

- **Structural:** class diagram, object diagram, component diagram, deployment diagram।
- **Behavioural:** use case diagram, sequence diagram, activity diagram, state machine diagram।

IDE থাকলে UML কেন লাগবে (CT-1)

IDE কোড থেকে ডায়াগ্রাম বানাতে পারে বা ক্লাসের কাঠামো তৈরি করে দিতে পারে, কিন্তু কোন ক্লাস থাকা উচিত, কার কী দায়িত্ব, সম্পর্কটা aggregation না composition — এই সিদ্ধান্তগুলো IDE নিতে পারে না। কোড থেকে বানানো ডায়াগ্রাম দেখায় কোড কেমন আছে, কেমন হওয়া উচিত তা নয়। UML দলের সবার মধ্যে যোগাযোগের ভাষা, বড় ছবিটা দেখায়, কোড লেখার আগে ভুল ধরতে সাহায্য করে এবং নতুন সদস্যদের দ্রুত বুঝতে সাহায্য করে। ডিজাইন ছাড়া বড় সিস্টেম বানাতে duplicate ক্লাস, god class, integration সমস্যা ও পরিবর্তনের উচ্চ খরচ দেখা দেয়। (Q&A: Q2)

2.7 Entity, Boundary and Control Objects

ডিজাইনের সময় object-গুলোকে তিন ভাগে ভাগ করলে দায়িত্ব পরিষ্কার থাকে:

ধরন	কাজ	AAS উদাহরণ
Entity	ডোমেইনের ধারণা ও ডেটা বহন করে, সাধারণত সংরক্ষিত থাকে	Field, Crop, SoilSensorReading
Boundary	সিস্টেম ও বাইরের actor বা বাইরের সিস্টেমের মধ্যে ইন্টারফেস	FarmerMobileScreen, WeatherAPIGateway
Control	একটি use case-এর কাজ সমন্বয় করে, business logic রাখে	IrrigationScheduler, HarvestPlanner

মনে রাখার কৌশল: Entity $\approx$ MVC-র Model, Boundary $\approx$ View, Control $\approx$ Controller। (Q&A: Q7)

2.8 Evolution of Programming Languages

1. **প্রাথমিক ভাষা (COBOL, Fortran):** প্রোগ্রাম ছিল লম্বা নির্দেশের তালিকা, ডেটা ছিল **global**। যেকোনো লাইন যেকোনো ভেরিয়েবল বদলাতে পারত, ফলে ডেটা নষ্ট হলে কোথায় নষ্ট হলো খুঁজে পাওয়া কঠিন ছিল — একে বলে **global data problem**।
2. **Procedural/block-structured ভাষা (ALGOL 68, Pascal, C):** function/procedure ও **local variable**, scope এল। প্রোগ্রামকে ফাংশনে ভাগ করা গেল, কিন্তু ডেটা আর সেই ডেটার উপর কাজ করা ফাংশন তখনও আলাদা ছিল।
3. **Abstract Data Type:** ডেটা স্ট্রাকচার ও তার উপর অনুমোদিত অপারেশনকে একসাথে রাখার ধারণা (যেমন stack-এর push/pop)।
4. **Object-oriented ভাষা (C++, Java):** ADT-কে ভাষার অংশ বানাল — **class**। ডেটা ও আচরণ একসাথে (encapsulation), private/public দিয়ে প্রবেশ নিয়ন্ত্রণ, inheritance ও polymorphism।

OOP যে সমস্যাগুলোর সমাধান করেছে: শেয়ার করা ডেটায় অনিয়ন্ত্রিত প্রবেশ, বাস্তব জগতের সাথে কোডের অমিল, কোড পুনর্ব্যবহারের অসুবিধা, এবং বড় প্রোগ্রামে পরিবর্তনের প্রভাব ছড়িয়ে পড়া। (Q&A: Q9)

2.9 Software Design vs Software Architecture

Software architecture হলো পুরো সিস্টেমের উচ্চ-স্তরের কাঠামো: কোন কোন বড় component বা subsystem থাকবে, তারা কীভাবে যুক্ত হবে, কোন architectural style (layered, client-server, MVC, microservices), কোন প্রযুক্তি। এই সিদ্ধান্তগুলো মূলত quality attribute (scalability, availability, security) দ্বারা চালিত এবং পরে বদলানো খুব ব্যয়বহুল।

Software design হলো প্রতিটি component-এর ভেতরের কাঠামো: ক্লাস, মেথড, সম্পর্ক, অ্যালগরিদম, design pattern। এটি তুলনামূলক নিম্ন-স্তরের এবং refactor করে বদলানো সহজ।

এক লাইনে মনে রাখা

Architecture হলো **শহরের নকশা** (কোথায় কোন ভবন, কোন রাস্তা); design হলো **প্রতিটি ভবনের ফ্লোর প্ল্যান**। DUMS উদাহরণ: Complaint, Billing, Housing আলাদা service হবে — architecture; Complaint service-এর ভেতরে Complaint ক্লাস, Observer দিয়ে notification — design। (Q&A: Q6)

2.10 Benefits of Modelling Real-World Entities as Objects

একটি object-এর থাকে **identity**, **state** (attribute) এবং **behaviour** (method) — ঠিক যেমন বাস্তব জগতের একটি সত্তার পরিচয়, বৈশিষ্ট্য ও কাজ থাকে। DUMS-এ Citizen, MunicipalService, Complaint-কে object হিসেবে মডেল করার সুবিধা:

1. **স্বাভাবিক মিল:** সিটি কর্পোরেশনের কর্মকর্তারা যে ভাষায় কথা বলেন, কোডেও সেই নাম — requirement থেকে কোড খুঁজে পাওয়া সহজ।
2. **Encapsulation:** complaint-এর status শুধু assign(), resolve() দিয়েই বদলানো যাবে, ফলে ভুল অবস্থাঠেকানো যায়।
3. **Modularity ও reuse:** একই Citizen ক্লাস complaint, billing, housing সব মডিউলে ব্যবহার হয়।
4. **Maintainability:** escalation-এর নিয়ম বদলালে শুধু সংশ্লিষ্ট ক্লাস বদলায়।
5. **Extensibility:** inheritance ও polymorphism দিয়ে নতুন সেবা যোগ করা যায় পুরনো কোড না ভেঙে। (Q&A: Q3)

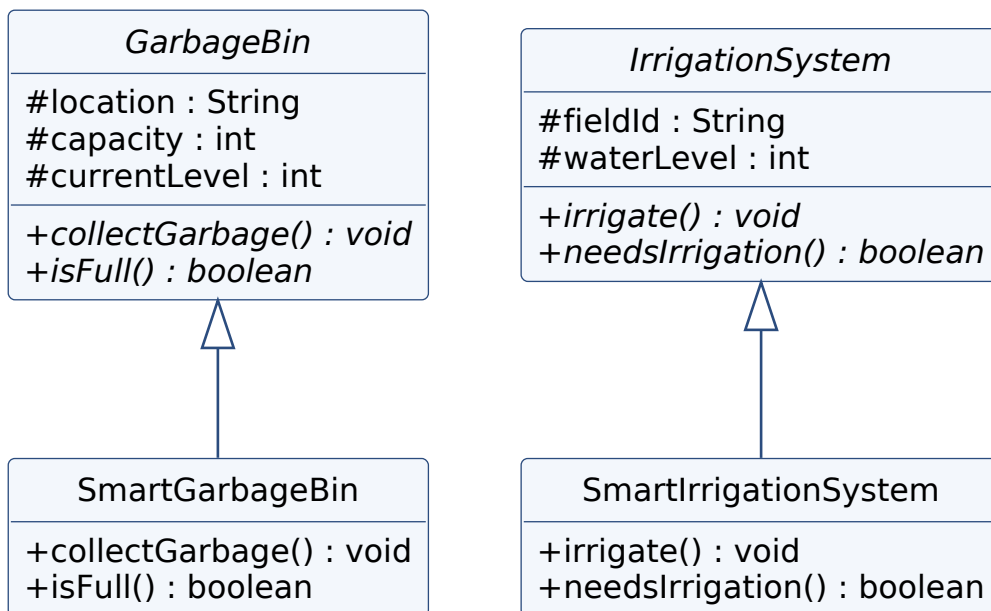
The Four Object-Oriented Design Principles

Slides অনুযায়ী ভালো OO ডিজাইনের চারটি মূল নীতি: **Abstraction, Encapsulation, Decomposition, Generalization**।

3.1 Abstraction

Abstraction মানে কোনো ধারণার শুধু সেই বৈশিষ্ট্যগুলো রাখা যা এই সমস্যার জন্য জরুরি, বাকি সব বাদ দেওয়া। Abstraction **context**-নির্ভর: DUMS-এর কাছে একটি ময়লার বিনের দরকারি তথ্য হলো অবস্থান, ধারণক্ষমতা আর বর্তমান ভরাট মাত্রা; কিন্তু বিন প্রস্তুতকারক কোম্পানির inventory সিস্টেমের কাছে দরকারি হলো উপাদান, রং, দাম। একই বাস্তব জিনিস, ভিন্ন context-এ ভিন্ন abstraction।

Java-তে abstraction প্রকাশের দুটি হাতিয়ার: **abstract class** (কিছু সাধারণ কোড ও কিছু abstract method) এবং **interface** (শুধু চুক্তি)। Abstract method বলে কী করতে হবে; concrete subclass ঠিক করে কীভাবে।



DUMS ও AAS-এ abstraction

```

1  abstract class GarbageBin {
2      protected String location;          // শুধু DUMS-এর জন্য দরকারি তথ্য
3      protected int capacity;
4      protected int currentLevel;
5      public abstract boolean isFull();    // কী করতে হবে তা বলা হলো
6      public abstract void collectGarbage(); // কীভাবে করবে তা subclass ঠিক করবে
7  }
8
9  class SmartGarbageBin extends GarbageBin {
10     @Override
11     public boolean isFull() { return currentLevel >= capacity * 0.9; } // ৯০% ভরলেই পূর্ণ ধরা হবে
12     @Override
13     public void collectGarbage() { currentLevel = 0; }                // খালি করা হলো
14 }
    
```

City dashboard শুধু `isFull()` ডাকে; সেন্সরটা ultrasonic না ওজন-ভিত্তিক তা জানার দরকার নেই। ফলে জটিলতা কমে এবং ভেতরের implementation বদলালেও ব্যবহারকারী ক্লাসে প্রভাব পড়ে না। (Q&A: Q10, Q11)

3.2 Encapsulation

Encapsulation-এর তিনটি দিক:

1. **Bundling:** ডেটা (attribute) এবং সেই ডেটার উপর কাজ করা মেথডকে একটি ক্লাসে একত্র করা।
2. **Data hiding / access control:** attribute-গুলো private রেখে শুধু নিয়ন্ত্রিত মেথডের (getter/setter) মাধ্যমে প্রবেশ দেওয়া, যাতে ভুল মান সেট করা না যায়।
3. **Black box আচরণ:** অন্য ক্লাস শুধু interface (মেথডের নাম, parameter, return) জানে; ভেতরের হিসাব-নিকাশ লুকানো থাকে। একে বলে **black-box encapsulation** (২০২৪ Q2(b))।

Food	Student
+groceryID : String +name : String +manufacturer : String +expiryDate : Date +price : double +isOnSale(date : Date) : boolean	-gpa : double -degreeProgram : String +getGPA() : double +setGPA(gpa : double) : void +getDegreeProgram() : String +setDegreeProgram(p : String) : void

Slides-এর Food ও Student ক্লাস

```

1 public class Student {
2     private double gpa;           // private: বাইরে থেকে সরাসরি বদলানো যাবে না
3     private String degreeProgram;
4
5     public double getGPA() { return gpa; }           // getter: শুধু পড়ার অনুমতি
6     public void setGPA(double gpa) {               // setter: যাচাই করে তবেই সেট
7         if (gpa < 0.0 || gpa > 4.0)
8             throw new IllegalArgumentException("invalid GPA");
9         this.gpa = gpa;
10    }
11    public String getDegreeProgram() { return degreeProgram; }
12    public void setDegreeProgram(String p) { this.degreeProgram = p; }
13 }

```

Access modifiers ও UML visibility

UML	Java	কোথা থেকে ব্যবহার করা যায়
+	public	যেকোনো জায়গা থেকে
#	protected	একই ক্লাস, একই package, subclass
~	default (package-private)	শুধু একই package
-	private	শুধু একই ক্লাসের ভেতরে

UML-এ static সদস্যের নিচে দাগ (underline) দেওয়া হয় এবং abstract ক্লাস বা মেথড italic-এ লেখা হয়।

Encapsulation বনাম Information hiding

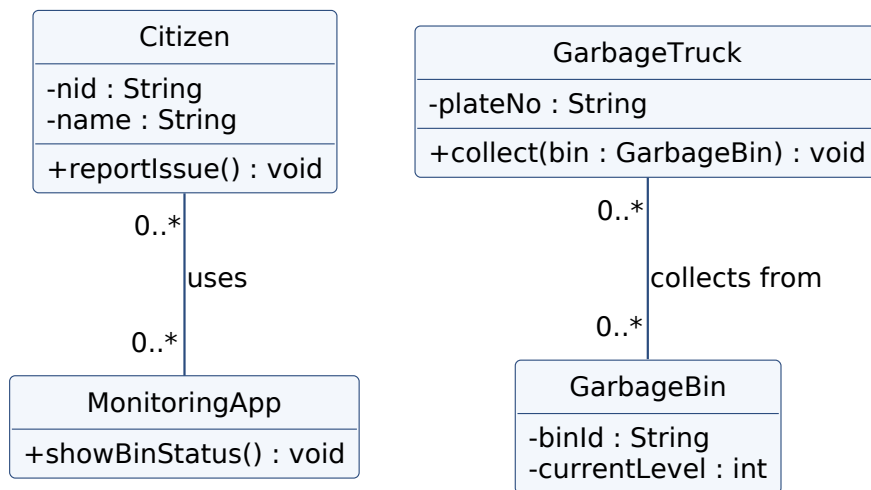
Encapsulation একটি কৌশল (ডেটা ও মেথড একসাথে বাঁধা ও প্রবেশ নিয়ন্ত্রণ); information hiding একটি নীতি (যে সিদ্ধান্তগুলো ভবিষ্যতে বদলাতে পারে সেগুলো স্থির interface-এর আড়ালে লুকিয়ে রাখা)। Encapsulation হলো information hiding অর্জনের একটি উপায়। সব field public রাখা ক্লাসও “encapsulated” কিন্তু কিছুই লুকায় না। (Q&A: Q12)

3.3 Decomposition

Decomposition মানে একটি জটিল সম্পূর্ণ জিনিসকে (whole) ছোট ছোট অংশে (parts) ভাগ করা, যাতে প্রতিটি অংশ আলাদাভাবে বোঝা, বানানো ও রক্ষণাবেক্ষণ করা যায়। অংশগুলোর সংখ্যা নির্দিষ্ট (fixed) বা পরিবর্তনশীল (dynamic) হতে পারে; অংশের আয়ু সম্পূর্ণের সাথে বাঁধা থাকতে পারে বা নাও পারে; অংশ অন্য whole-এর সাথে শেয়ার হতে পারে বা নাও পারে। এই পার্থক্য থেকেই তিন ধরনের সম্পর্ক।

3.3.1 Association (“uses-a / knows-a”)

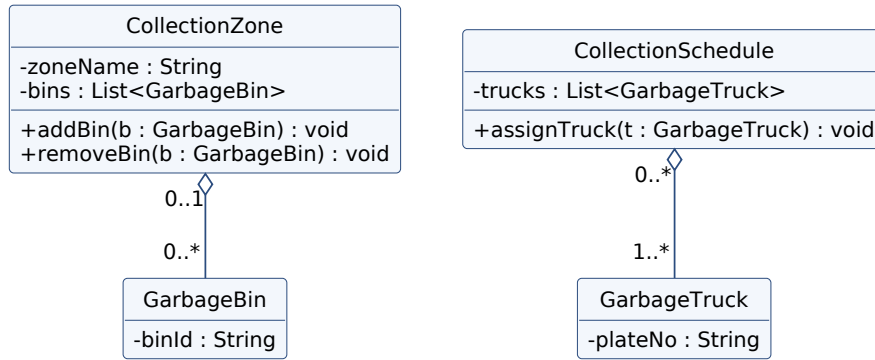
দুটি object একে অপরকে চেনে বা ব্যবহার করে, কিন্তু **কেউ কারও মালিক নয়**; দুজনের আয়ু স্বাধীন। UML-এ সাধারণ রেখা (দিক বোঝাতে তীর), দুই পাশে multiplicity। উদাহরণ: ছাত্র খেলাধুলা করে; DUMS-এ garbage truck অনেক bin থেকে ময়লা তোলে, একটি bin অনেক truck খালি করে।



DUMS-এ association

3.3.2 Aggregation (দুর্বল “has-a”)

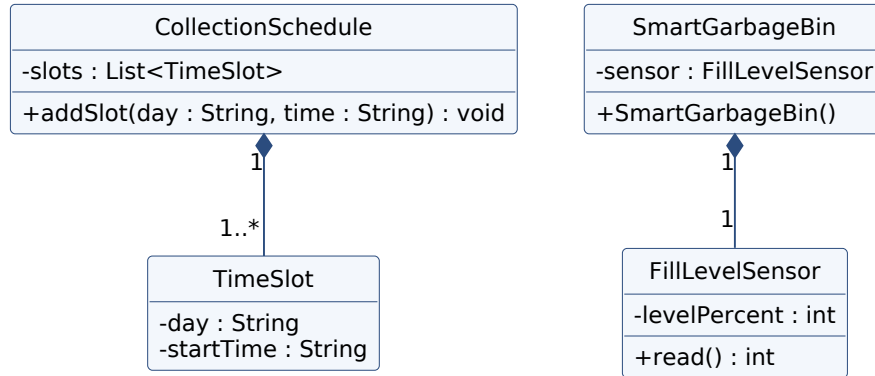
Whole-এর মধ্যে part থাকে, কিন্তু **part whole ছাড়াও টিকে থাকতে পারে** এবং অন্য whole-এ চলে যেতে বা শেয়ার হতে পারে। UML-এ whole-এর দিকে **ফাঁপা হীরা** (◊)। উদাহরণ: বিমানের crew — বিমান বাতিল হলেও crew থাকে। DUMS-এ collection zone-এ bin থাকে, কিন্তু bin অন্য zone-এ সরানো যায়।



DUMS-এ aggregation

3.3.3 Composition (শক্তিশালী “part-of”)

Whole তার part-এর মালিক: part whole ছাড়া থাকতে পারে না, শেষার হয় না, whole ধ্বংস হলে part-ও ধ্বংস হয়। UML-এ whole-এর দিকে **ভরাট হীরা (◆)**। উদাহরণ: বাড়ি ও তার ঘর; DUMS-এ smart bin-এর built-in fill-level sensor।



DUMS-এ composition

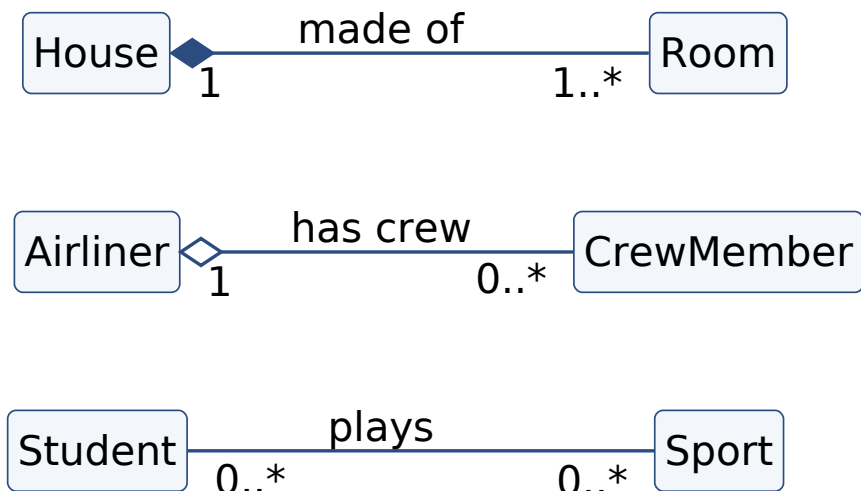
```

1 class GarbageTruck {                                     // Association
2     public void collect(GarbageBin bin) { bin.empty(); } // শুধু ব্যবহার করে, মালিক নয়
3 }
4
5 class CollectionZone {                                    // Aggregation
6     private List<GarbageBin> bins = new ArrayList<>();
7     public void addBin(GarbageBin b) { bins.add(b); } // bin বাইরে তৈরি হয়ে ভেতরে আসে
8     public void removeBin(GarbageBin b) { bins.remove(b); } // সরালেও bin টিকে থাকে
9 }
10
11 class SmartGarbageBin {                                   // Composition
12     private final FillLevelSensor sensor;
13     public SmartGarbageBin() {
14         sensor = new FillLevelSensor(); // ভেতরেই তৈরি, bin-এর সাথেই মরে যাবে
15     }
16 }
  
```

	Association	Aggregation	Composition
অর্থ	ব্যবহার করে/চেনে	আছে (দুর্বল)	অংশ (শক্তিশালী)
আয়ু	স্বাধীন	part টিকে থাকে	whole-এর সাথে শেষ
শেয়ার	–	হতে পারে	কখনো না
UML	সাধারণ রেখা	ফাঁপা হীরা	ভরাট হীরা
Java-র চিহ্ন	parameter/reference	বাইরে থেকে পাঠানো	constructor-এ new

দুটি দ্রুত পরীক্ষা

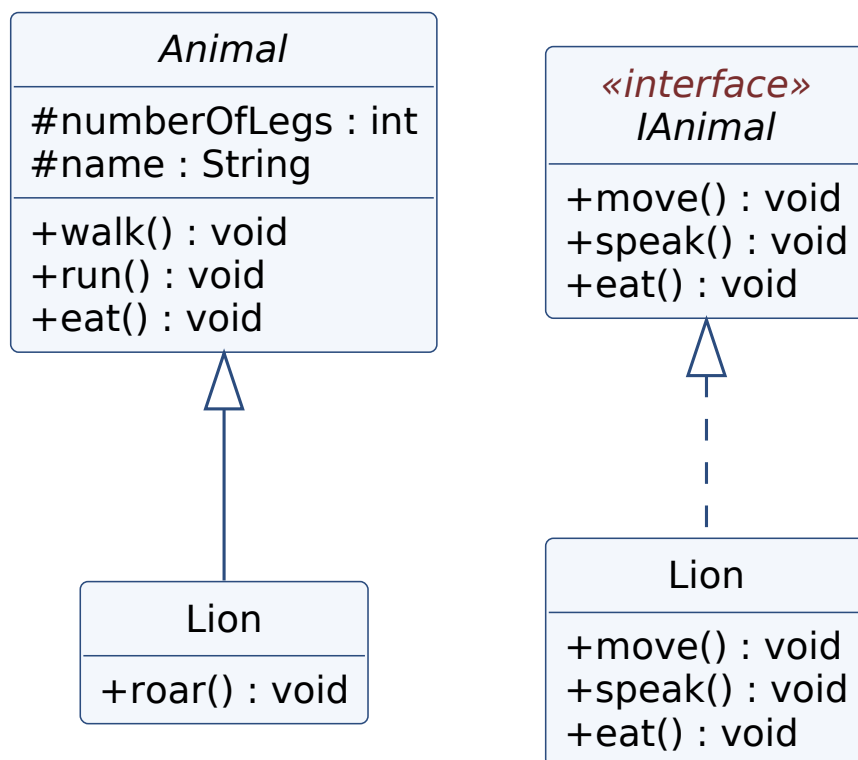
“Whole মুছে গেলে part-এর কি অর্থ থাকে?” – থাকলে aggregation। “Part কি অন্য whole-এ সরানো যায়?” – গেলে aggregation। নাহলে composition। **Multiplicity:** 1, 0..1, 0..* (বা *), 1..*, m..n। (Q&A: Q13, Q14, Q15)



Slides-এর সাধারণ উদাহরণ

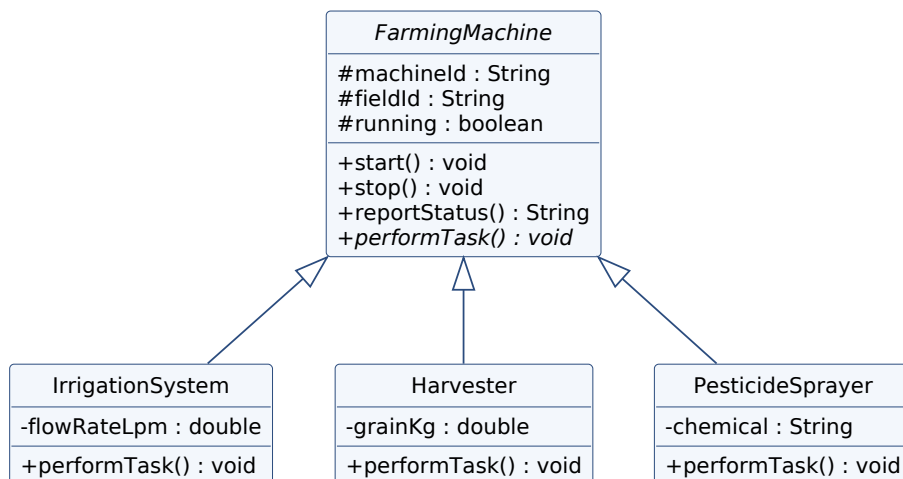
3.4 Generalization and Inheritance

Generalization মানে একাধিক ক্লাসের সাধারণ attribute ও behaviour একটি সাধারণ (general) superclass-এ তুলে নেওয়া; subclass-গুলো সেগুলো উত্তরাধিকারসূত্রে পায় (**inheritance**) এবং নিজের বিশেষ বৈশিষ্ট্য যোগ করে বা মেথড override করে। এটি “**is-a**” সম্পর্ক: IrrigationSystem is a FarmingMachine। মূল লক্ষ্য DRY (Don’t Repeat Yourself)। UML-এ generalization দেখানো হয় subclass থেকে superclass-এর দিকে **ফাঁপা ত্রিভুজ মাথার সলিড তীর** দিয়ে; interface implement করা (realization) দেখানো হয় **ফাঁপা ত্রিভুজ মাথার ড্যাশ তীর** দিয়ে।



Class inheritance & interface inheritance

3.4.1 AAS-9 generalization



FarmingMachine hierarchy

```

1 abstract class FarmingMachine {
2     protected final String machineId;    // সব মেশিনের সাধারণ তথ্য একবারই লেখা
3     protected boolean running;
4     protected FarmingMachine(String id) { machineId = id; }
5     public void start() { running = true; } // সাধারণ আচরণ, সবাই পারে
6     public abstract void performTask();    // প্রত্যেক মেশিন নিজের মতো করবে
7 }
8

```

```

9  class Harvester extends FarmingMachine {
10     Harvester(String id) { super(id); }           // parent-এর constructor পুনর্ব্যবহার
11     @Override
12     public void performTask() { System.out.println(machineId + " harvesting"); } // override
13 }

```

সুবিধা: Reusability (সাধারণ কোড একবার), Maintainability (এক জায়গায় বদলালেই সব জায়গায় কার্যকর), Extensibility (নতুন SeedPlanter শুধু extend করলেই হবে), Polymorphism (একটি লুপে সব মেশিন চালানো)। আলাদা আলাদা ক্লাসে একই কোড লিখলে duplicate কোড, অসঙ্গত আচরণ ও বাড়তি খরচ হয়। (Q&A: Q16)

3.4.2 Interface inheritance & multiple inheritance

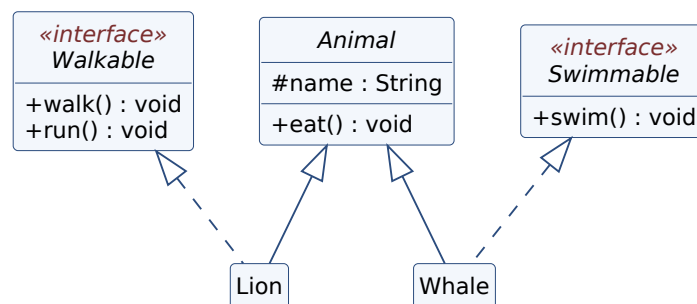
Java-তে একটি ক্লাস একটির বেশি ক্লাস extend করতে পারে না, কারণ দুই parent-এ একই নামের মেথড থাকলে কোনটা চলবে তা অস্পষ্ট হয় (**diamond problem**)। সমাধান: **একটি ক্লাস extend + একাধিক interface implement**। Interface শুধু সক্ষমতার চুক্তি (যেমন Flyable, Sprayer); implementation ক্লাস নিজেই দেয়, তাই দ্বন্দ্ব হয় না। (Q&A: Q19)

3.4.3 Inheritance-এর ভুল ব্যবহার ও Liskov Substitution Principle

LSP: “subclass-এর object যেকোনো জায়গায় superclass-এর object-এর বদলে বসানো গেলে প্রোগ্রাম ঠিকভাবে চলতে হবে” অর্থাৎ subclass-কে parent-এর প্রতিটি প্রতিশ্রুতি পালন করতে হবে।

- **Whale extends Animal (২০২৩):** Animal-এ walk(), run() আছে যা সব প্রাণীর সাধারণ নয়। Whale এগুলো override করে ভেতরে swim() ডাকে — আচরণটা বিভ্রান্তিকর, numberOfLegs = 0 একটা কৌশল মাত্র, আর swim() private হওয়ায় আসল ক্ষমতা লুকানো। এটি LSP ভঙ্গ ও “refused bequest” code smell।
- **SolarIrrigationPump.refuel():** parent ধরে নিয়েছে সব মেশিন জ্বালানি নেয়; solar pump UnsupportedOperationException ছোড়ে, ফলে কোড compile হয় কিন্তু runtime-এ ক্র্যাশ করে।

সমাধান: parent-এ শুধু সত্যিকারের সাধারণ আচরণ রাখা, ঐচ্ছিক ক্ষমতাগুলো interface-এ নাও (Walkable, Swimmable, FuelPowered, SolarPowered)।



Whale সমস্যার সঠিক ডিজাইন (Q&A: Q17, Q18)

3.5 UML Class Diagram Notation Cheat-Sheet

সম্পর্ক	চিহ্ন	কখন ব্যবহার
Association	সাধারণ সলিড রেখা	object-গুলো একে অপরকে চেনে
Directed association	সলিড রেখা + খোলা তীর	এক দিকেই চেনে ($A \rightarrow B$)
Aggregation	ফাঁপা হীরা whole-এর দিকে	দুর্বল whole-part
Composition	ভরাট হীরা whole-এর দিকে	শক্তিশালী whole-part
Generalization	সলিড রেখা + ফাঁপা ত্রিভুজ	class inheritance (extends)
Realization	ড্যাশ রেখা + ফাঁপা ত্রিভুজ	interface implement
Dependency	ড্যাশ রেখা + খোলা তীর	সাময়িক ব্যবহার (parameter, local variable)

ক্লাস বক্সের তিনটি ভাগ: উপরে নাম, মাঝে attribute (- name : Type), নিচে operation (+ method(p : Type) : ReturnType)

Design Complexity and Code Quality

4.1 Coupling

Coupling হলো দুটি মডিউলের পারস্পরিক নির্ভরতার মাত্রা — একটি মডিউল অন্যটির ভেতরের বিষয়ের উপর কতটা নির্ভরশীল। **Tight coupling** থাকলে একটিতে পরিবর্তন করলে অন্যটিতেও পরিবর্তন করতে হয়। লক্ষ্য হলো **loose coupling**।

Slides অনুযায়ী coupling মূল্যায়নের তিনটি মানদণ্ড:

1. **Degree**: একটি মডিউলের সাথে অন্য মডিউলের সংযোগের সংখ্যা ও আকার। কম parameter, কম সংযোগ ভালো।
2. **Ease**: সংযোগগুলো কতটা স্পষ্ট। ভেতরের implementation না জেনেই সহজে যুক্ত হওয়া যাওয়া উচিত। “flag ১ মানে road” — এ ধরনের লুকানো অর্থ ease কমায়।
3. **Flexibility**: অন্য মডিউলগুলো কতটা বদলযোগ্য। Loose coupling থাকলে একটি implementation সরিয়ে অন্যটি বসানো যায়।

Coupling-এর ধরন (চিলা থেকে শক্ত): data coupling → stamp coupling → **control coupling** (flag পাঠিয়ে অন্য মডিউলের ভেতরের কাজ নিয়ন্ত্রণ) → common coupling (global data) → content coupling (অন্যের ভেতরের ডেটা সরাসরি বদলানো)।

4.2 Cohesion

Cohesion হলো একটি মডিউল কতটা স্পষ্টভাবে একটি নির্দিষ্ট কাজে মনোযোগী। High cohesion মানে ক্লাসের সব attribute ও মেথড একটি উদ্দেশ্যে একসাথে কাজ করে। একটি ক্লাস যদি অনেকগুলো সম্পর্কহীন কাজ করে, তাহলে low cohesion — বোঝা, পুনর্ব্যবহার ও পরিবর্তন কঠিন।

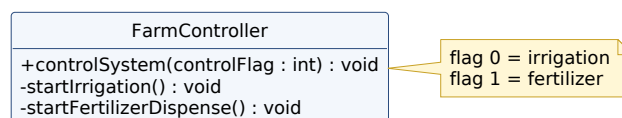
Cohesion-এর স্তর (ভালো থেকে খারাপ): functional → sequential → communicational → procedural → temporal → **logical** (flag দিয়ে বাছাই করা সম্পর্কহীন কাজ) → coincidental।

Coupling ও cohesion-এর সম্পর্ক

ভালো ডিজাইন = **loose coupling + high cohesion**। সাধারণত একটি low-cohesion ক্লাস ভাগ করলে flag-ভিত্তিক tight coupling-ও দূর হয়। তবে অতিরিক্ত ভাগ করলে অসংখ্য ছোট ক্লাস আর অনেক সংযোগ তৈরি হয় — তাই ভারসাম্য দরকার।

4.3 The Control-Flag Problem: FarmController

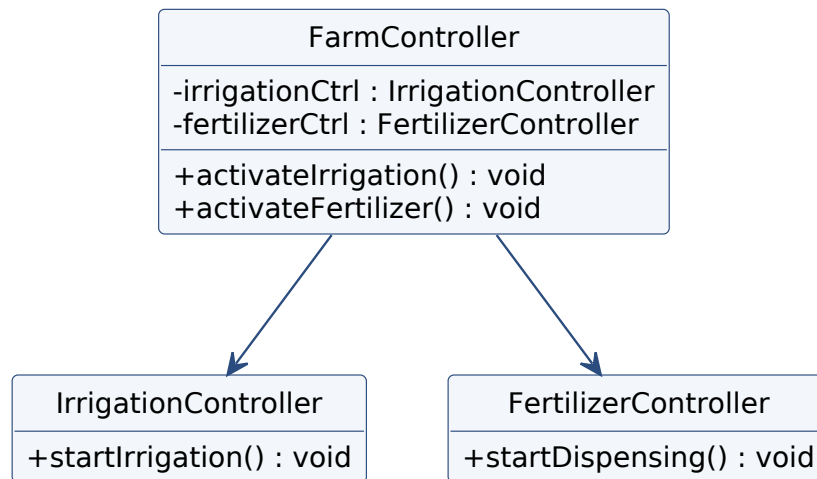
Slides-এর উদাহরণে `FarmController.controlSystem(int controlFlag)` flag ০ পেলে সেচ শুরু করে, ১ পেলে সার দেয়।



খারাপ ডিজাইন: flag-নির্ভর একটি মেথড

- **Cohesion কম**: সেচ আর সার দেওয়া দুটি আলাদা দায়িত্ব এক ক্লাসে।
- **Coupling বেশি (control coupling)**: যে ক্লাস ডাকছে তাকে জানতে হয় ০ আর ১-এর অর্থ; flag-এর মান বদলালে বা নতুন কাজ যোগ হলে caller-দেরও বদলাতে হয়; ভুল flag runtime-এ গিয়ে ধরা পড়ে।

সমাধান: প্রতিটি কাজের জন্য আলাদা cohesive ক্লাস, আর `FarmController` শুধু স্পষ্ট নামের মেথড দিয়ে সমন্বয় করবে।



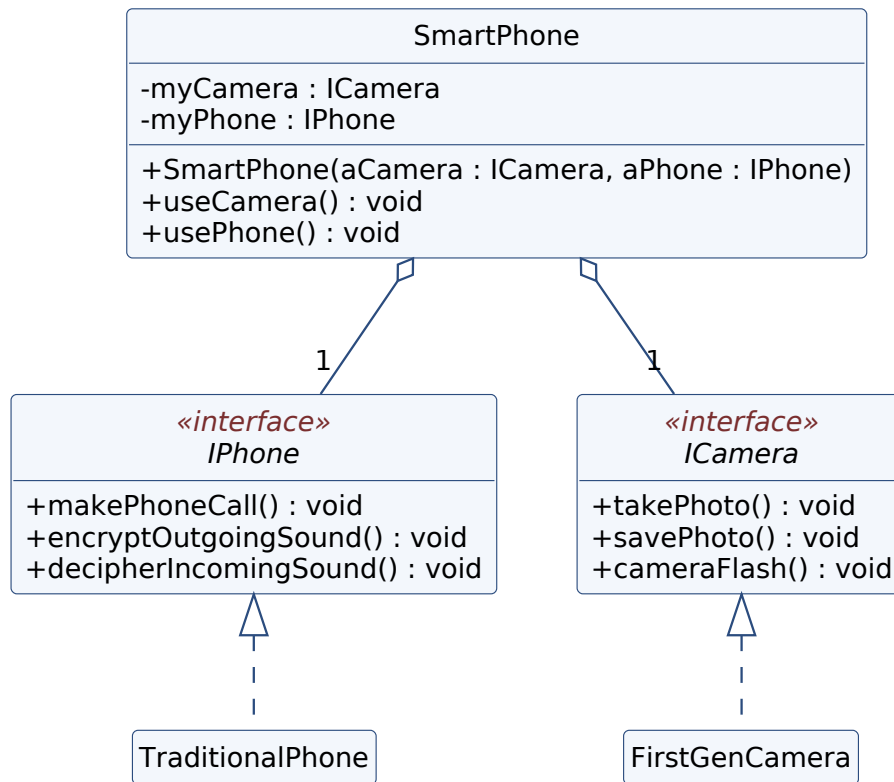
ভালো ডিজাইন: আলাদা controller

Slides-এর শেষ exercise-এ ঠিক একই সমস্যা DUMS-এর `ServiceManager.processRequest(int flag)`-এ দেওয়া আছে (waste collection ও road maintenance)। এটি পরীক্ষায় আসার সম্ভাবনা খুব বেশি — পূর্ণ সমাধান Q&A: Q21-এ। একই ধরনের পুরনো প্রশ্ন: ২০২৩-এর Sensor (Q&A: Q22)।

4.4 Separation of Concerns

Separation of Concerns (SoC) মানে সিস্টেমকে এমনভাবে ভাগ করা যাতে প্রতিটি অংশ একটি নির্দিষ্ট “concern” (উদ্বেগ/দায়িত্ব) নিয়ে কাজ করে। SoC প্রয়োগের ধাপ: (১) concern-গুলো চিহ্নিত করো; (২) প্রতিটির জন্য আলাদা interface/ক্লাস বানাও; (৩) মূল ক্লাসটিকে সমন্বয়কারী বানাও যা ওই অংশগুলো ধারণ করে ও কাজ ভাগ করে দেয়।

SmartPhone উদাহরণ (২০২১): একটি ক্লাসে ক্যামেরা (ছবি তোলা, সেভ, ফ্ল্যাশ) আর ফোন (কল, এনক্রিপ্ট, ডিক্রিপ্ট) দুটো concern মেশানো — low cohesion ও tight coupling; ক্যামেরা বদলালে ফোনের কোডও ঝুঁকিতে পড়ে। SoC প্রয়োগ করে `ICamera` ও `IPhone` interface, আলাদা implementation, আর `SmartPhone` constructor-এ এগুলো গ্রহণ করে।



SoC প্রয়োগের পর SmartPhone (Q&A: Q23)

```

1 public class SmartPhone {
2     private final ICamera myCamera;    // ক্যামেরা concern আলাদা
3     private final IPhone myPhone;      // ফোন concern আলাদা
4
5     public SmartPhone(ICamera aCamera, IPhone aPhone) { // বাইরে থেকে অংশগুলো আসে
6         this.myCamera = aCamera;
7         this.myPhone = aPhone;
8     }
9     public void useCamera() { myCamera.takePhoto(); } // কাজ ভাগ করে দেওয়া (delegation)
10    public void usePhone() { myPhone.makePhoneCall(); }
11 }

```

Slides-এর কোডে ভুল

Slides-এ void মেথডের ভেতরে `return this.myCamera.takePhoto();` লেখা আছে — এটি compile হবে না। `return` ছাড়া শুধু মেথড কল করো।

4.5 Information Hiding

Information hiding (David Parnas) নীতি বলে: যে ডিজাইন সিদ্ধান্তগুলো ভবিষ্যতে বদলাতে পারে (ডেটা কীভাবে রাখা হয়, কোন অ্যালগরিদম, কোন হার্ডওয়্যার), সেগুলো একটি মডিউলের ভেতরে লুকিয়ে রাখা এবং বাইরে শুধু একটি স্থির interface দাও। এতে পরিবর্তন হলে তার প্রভাব এক মডিউলেই সীমাবদ্ধ থাকে। Java-তে access modifier আর interface এর প্রধান হাতিয়ার। যেমন DUMS-এর `PaymentProcessor` interface — ভেতরে ব্যাংক কার্ড না মোবাইল ওয়ালেট, বাকি সিস্টেম জানে না।

4.6 Conceptual Integrity

Conceptual integrity মানে পুরো সফটওয়্যারের ডিজাইনে ধারাবাহিকতা ও সামঞ্জস্য — অনেক মানুষ মিলে বানাতেও মনে হবে যেন একজন মানুষের মস্তিষ্ক থেকে এসেছে। একই ধারণা, একই নামকরণ, একই প্যাটার্ন, একই স্টাইল সব জায়গায়। Fred Brooks একে সিস্টেম ডিজাইনের সবচেয়ে গুরুত্বপূর্ণ বিবেচ্য বলেছেন। এমন সিস্টেম শেখা, ব্যবহার করা ও রক্ষণাবেক্ষণ করা সহজ।

অর্জনের উপায়

1. দলের মধ্যে যোগাযোগ: code review, agile চর্চা যেমন daily stand-up ও sprint retrospective।
2. একই design principle ও programming method সবাই অনুসরণ করা।
3. সুসংজ্ঞায়িত architecture যা সব মডিউল মেনে চলে।
4. একটি ছোট মূল দল বা chief architect যারা ডিজাইন সিদ্ধান্ত ও commit অনুমোদন করে।
5. নামকরণের নিয়ম ও coding standard।
6. একই ধরনের সমস্যায় একই design pattern।
7. শেয়ার করা ডকুমেন্টেশন/UML ও নিয়মিত refactoring।

DUMS উদাহরণ: সব মডিউলে একই Complaint status মান, একই Observer-ভিত্তিক notification, একই REST নামকরণ।
Integrity না থাকলে: billing বলে customerId, complaint বলে citizenId; এক মডিউল push notification দেয়, অন্যটি প্রতি মিনিটে ডেটাবেস poll করে। (Q&A: Q24)

4.7 Code Smells

Code smell হলো কোডের এমন লক্ষণ যা সরাসরি bug নয়, কিন্তু খারাপ ডিজাইনের ইঙ্গিত দেয় এবং ভবিষ্যতে সমস্যা তৈরি করবে। Smell পাওয়া গেলে **refactoring** (আচরণ না বদলে কোডের গঠন উন্নত করা) করতে হয়।

Smell	লক্ষণ	প্রতিকার
Long Method	একটি মেথড অনেক কাজ করে	Extract Method
Large Class / God Class	একটি ক্লাস অনেক দায়িত্ব	Extract Class (SRP)
Duplicated Code	একই কোড একাধিক জায়গায়	Extract Method, Pull Up
Long Parameter List	অনেক parameter	Parameter Object, Builder
Feature Envy	মেথড নিজের ক্লাসের চেয়ে অন্য ক্লাসের ডেটায় বেশি আগ্রহী	Move Method
Switch Statements	টাইপ দেখে switch/if-else	Polymorphism
Message Chains	a.getB().getC().getD()	Hide Delegate
Refused Bequest	subclass উত্তরাধিকার পাওয়া মেথড প্রত্যাখ্যান করে	interface/composition
Shotgun Surgery	এক পরিবর্তনে অনেক ক্লাস বদলাতে হয়	Move Method/Field
Divergent Change	এক ক্লাস অনেক কারণে বদলায়	Extract Class
Comments	অস্পষ্ট কোড চাকতে কमेंট	নাম বদলানো, Extract Method
Middle Man	ক্লাস শুধু কাজ অন্যকে পাঠায়	Remove Middle Man
Data Class	শুধু getter/setter, কোনো আচরণ নেই	আচরণ ক্লাসে সরানো

২০২৪-এর OrderProcessor

OrderProcessor.process() পরপর order.calculateTotal(), applyDiscount(), generateInvoice() ডাকে — এটি **Feature Envy**: checkout-এর ধাপ ও তাদের ক্রম Order-এর জ্ঞান, কিন্তু সেটা বাইরে আছে। নতুন ধাপ (tax) যোগ হলে সব caller বদলাতে হবে, আর কেউ ভুল ক্রমে ডাকতে পারে। সমাধান: Order-এর ভেতরে checkout() মেথড (Move

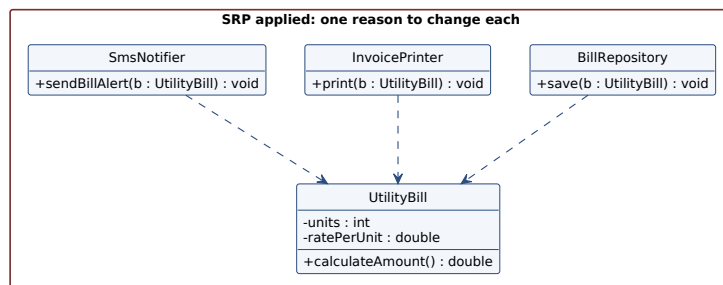
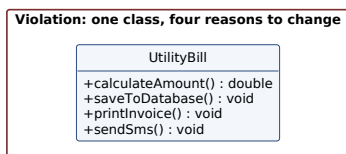
Method, “Tell, don’t ask”)। (Q&A: Q25)

Comment কখন smell

কমেন্ট নিজে খারাপ নয়; কিন্তু অস্পষ্ট কোড ব্যাখ্যা করতে কমেন্ট লাগলে সেটা smell। যেমন `int d; // elapsed days` (নাম দাও `elapsedDays`), লম্বা মেথডে `// validate, // save` ব্লক (আলাদা মেথড বানাও), পুরনো ভুল কমেন্ট, `comment-out` করা মৃত কোড। ভালো কমেন্ট ব্যাখ্যা করে কেন একটা সিদ্ধান্ত নেওয়া হয়েছে।

4.8 SOLID Principles and the Law of Demeter

Principle	অর্থ ও উদাহরণ
S – Single Responsibility	একটি ক্লাস বদলানোর কারণ একটাই হবে। <code>UtilityBill</code> শুধু হিসাব করবে; ডেটাবেসে রাখা, প্রিন্ট, SMS আলাদা ক্লাসে। (Q&A: Q26)
O – Open/Closed	ক্লাস extension-এর জন্য খোলা, modification-এর জন্য বন্ধ। Toll-এর switch বাদ দিয়ে polymorphism – নতুন <code>Ambulance</code> ক্লাস যোগ, পুরনো কোড অপরিবর্তিত। (Q&A: Q27)
L – Liskov Substitution	subclass-কে parent-এর জায়গায় বসালে প্রোগ্রাম ঠিক চলবে (<code>Whale</code> , <code>SolarPump</code>)।
I – Interface Segregation	বড় একটি interface-এর বদলে ছোট ছোট নির্দিষ্ট interface (<code>Flyable</code> , <code>Sprayer</code>)।
D – Dependency Inversion	উচ্চ-স্তরের মডিউল নিম্ন-স্তরের concrete ক্লাসের উপর নয়, abstraction-এর উপর নির্ভর করবে (<code>ServiceManager</code> → <code>MunicipalService</code>)।
Law of Demeter	“শুধু নিকটতম বন্ধুর সাথে কথা বলো” – একটি মেথড শুধু নিজের object, parameter, নিজে তৈরি করা object ও নিজের field-এর মেথড ডাকবে। <code>c.getCitizen().getAddress().getZone().getOfficer()</code> এটি ভঙ্গ করে। (Q&A: Q28)



SRP ভঙ্গ বনাম SRP প্রয়োগ

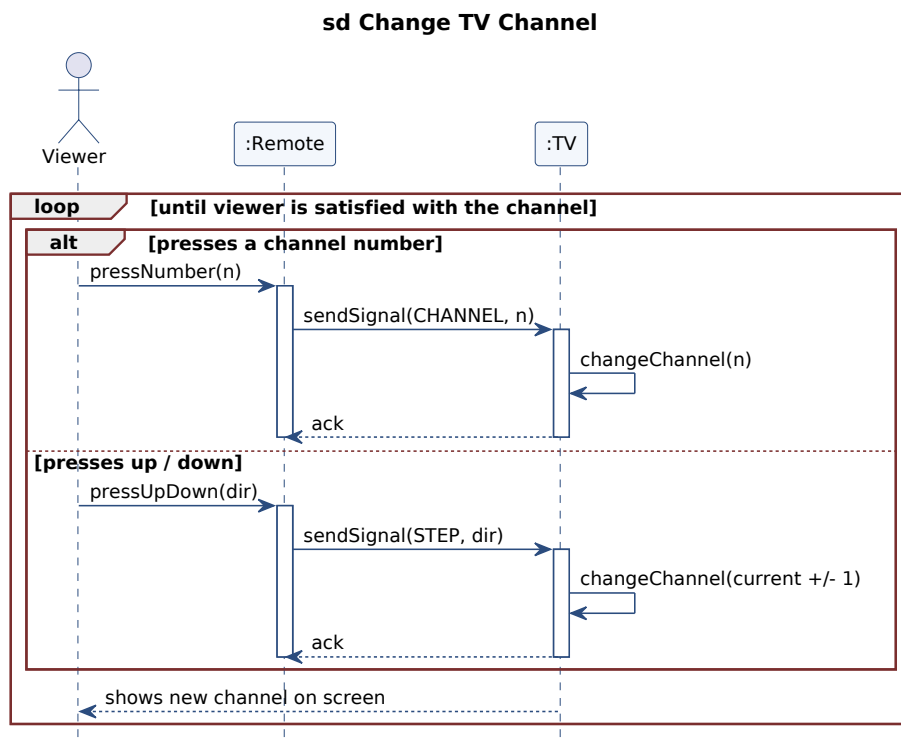
Behavioural Modelling with UML

5.1 UML Sequence Diagram

Sequence diagram দেখায় একটি নির্দিষ্ট সিনারিওতে object-গুলো **সময়ের ক্রমে** কীভাবে message আদান-প্রদান করে। সময় উপর থেকে নিচে চলে।

উপাদান	অর্থ ও চিহ্ন
Frame (sd)	পুরো ডায়াগ্রামকে ঘিরে রাখা আয়তক্ষেত্র, উপরে বাঁ কোণে নাম
Actor	সিস্টেমের বাইরের ব্যবহারকারী (কাঠি-মানুষ)
Lifeline	object (:TV) ও তার নিচে ড্যাশ রেখা
Activation bar	lifeline-এর উপর সুরু আয়তক্ষেত্র — object তখন ব্যস্ত
Synchronous message	সলিড রেখা, ভরাট তীর — উত্তরের জন্য অপেক্ষা করে
Asynchronous message	সলিড রেখা, খোলা তীর — অপেক্ষা করে না
Return message	ড্যাশ রেখা, খোলা তীর
Self message	একই lifeline-এ ফিরে আসা তীর
Create message	নতুন object তৈরি; তার lifeline ওখান থেকে শুরু
alt fragment	if-else; প্রতিটি অংশে guard [condition]
opt fragment	শুধু if (শর্ত সত্য হলে)
Loop fragment	পুনরাবৃত্তি, guard-সহ

TV রিমোট দিয়ে চ্যানেল বদলানো (২০২১, ২০২৪)



TV-remote sequence diagram

ব্যাখ্যা: দর্শক সন্তুষ্ট না হওয়া পর্যন্ত loop চলে। ভেতরে alt: নম্বর চাপলে রিমোট TV-কে `sendSignal(CHANNEL, n)` পাঠায় এবং TV নিজেকে `changeChannel(n)` ডাকে; অন্যথায় up/down চাপলে পরের বা আগের চ্যানেলে যায়। শেষে TV নতুন চ্যানেল দেখায়। (Q&A: Q29; DUMS অভিযোগ সিনারিও Q30)

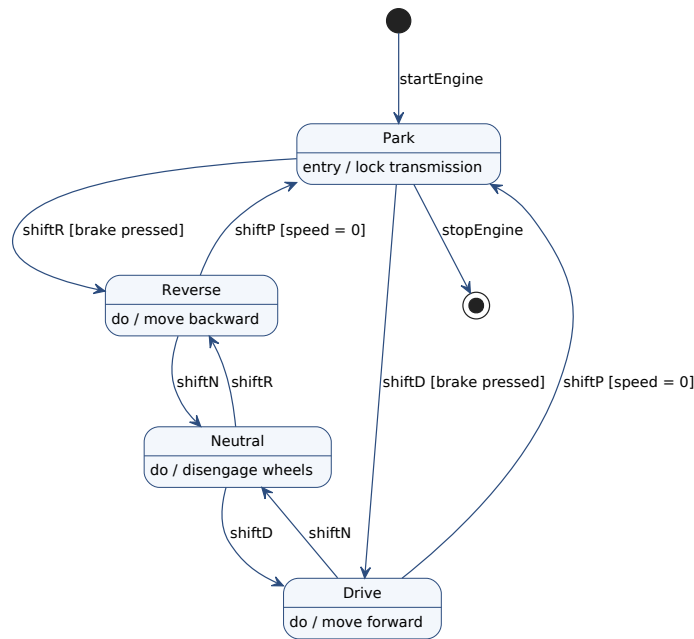
Sequence diagram আঁকার নিয়ম

(১) actor বাঁ দিকে, তারপর boundary, control, entity ক্রমে lifeline। (২) প্রতিটি message-এ মেথডের নাম ও parameter। (৩) শর্তে alt/opt, পুনরাবৃত্তিতে loop, guard বন্ধনীতে। (৪) return message ড্যাশ দিয়ে। (৫) frame-এ sd নাম দাও।

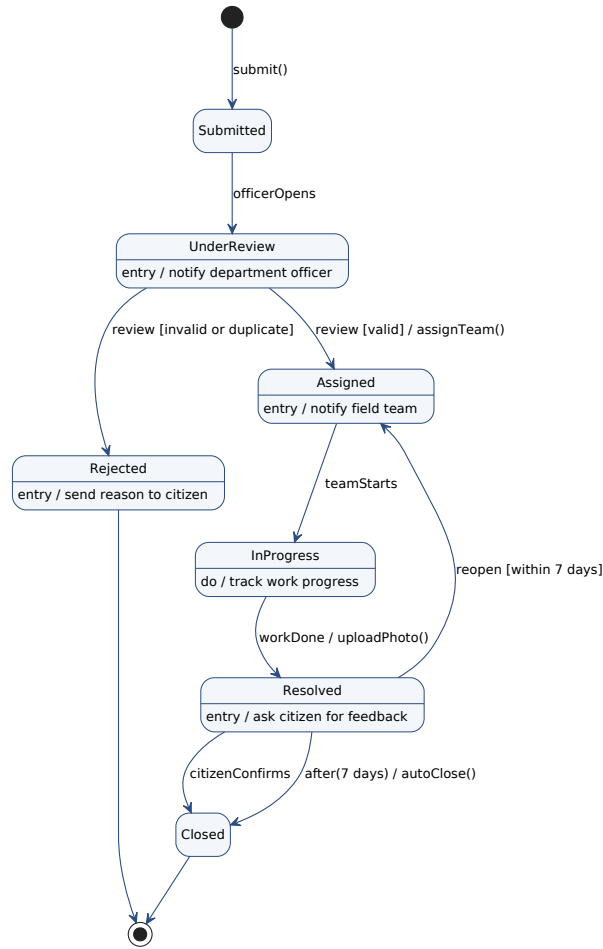
5.2 UML State Diagram

State (machine) diagram দেখায় একটি object তার জীবনকালে বিভিন্ন event-এর প্রতিক্রিয়ায় কীভাবে এক অবস্থা থেকে আরেক অবস্থায় যায়।

- **State:** গোলাকার কোণওয়ালা আয়তক্ষেত্র — object-এর একটি অবস্থা।
- **Initial pseudo-state:** ভরাট কালো বৃত্ত; **Final state:** বৃত্তের ভেতরে ভরাট বৃত্ত।
- **Transition:** তীর, লেবেল event [guard] / action — event ঘটলে, guard সত্য হলে transition হবে এবং action চলবে।
- **Internal activity:** entry / (চোকার সময়), do / (অবস্থায় থাকাকালীন), exit / (বের হওয়ার সময়)।



গাড়ির transmission-এর অবস্থা



DUMS complaint-এর অবস্থা (Q&A: Q31)

5.3 Model Checking

Model checking হলো একটি ডিজাইন মডেলকে (যেমন state machine) প্রয়োজনীয় বৈশিষ্ট্যের বিপরীতে স্বয়ংক্রিয়ভাবে যাচাই করার কৌশল, যেখানে মডেলের সম্ভাব্য প্রতিটি অবস্থা পরীক্ষা করা হয়। তিনটি ধাপ:

1. **Modelling phase:** সিস্টেমকে state model হিসেবে বর্ণনা করা এবং যাচাই করার বৈশিষ্ট্য লেখা, যেমন “গাড়ি Drive থেকে সরাসরি Reverse-এ যাবে না” (safety) বা “প্রতিটি complaint শেষ পর্যন্ত Closed বা Rejected হবে” (liveness)।
2. **Running phase:** model checker সব অবস্থা ও transition ঘুরে দেখে।
3. **Analysis phase:** কোনো বৈশিষ্ট্য ব্যর্থ হলে টুল একটি **counterexample** দেয় — ঠিক কোন event-এর ক্রমে ভুলটা ঘটে — যা দেখে ডিজাইন ঠিক করা হয়।

সুবিধা: কোড লেখার আগেই deadlock, অপ্রাপ্য অবস্থা, অনিরাপদ transition ধরা পড়ে; testing যেখানে কিছু পথ দেখে, model checking সব পথ দেখে। (Q&A: Q32)

Design Pattern Foundations

6.1 What is a Design Pattern

Software design pattern হলো সফটওয়্যার ডিজাইনে বারবার দেখা দেওয়া একটি সমস্যার **সাধারণ, পুনর্ব্যবহারযোগ্য ও পরীক্ষিত সমাধান**। এটি সরাসরি কপি করার মতো তৈরি কোড বা লাইব্রেরি নয়; বরং একটি টেমপ্লেট যা বলে কোন কোন ক্লাস/object অংশ নেবে, তাদের ভূমিকা কী, আর তারা কীভাবে একসাথে কাজ করবে। ধারণাটি এসেছে স্থপতি Christopher Alexander-এর কাছ থেকে; ১৯৯৪ সালে Gamma, Helm, Johnson ও Vlissides — যাঁদের **Gang of Four (GoF)** বলা হয় — ২৩টি object-oriented pattern-এর তালিকা প্রকাশ করেন।

প্রতিটি pattern-এর চারটি মূল উপাদান:

1. **Pattern name:** এক-দুই শব্দে পুরো সমাধানের নাম — দলের মধ্যে সাধারণ ভাষা তৈরি করে।
2. **Problem:** কখন pattern-টি প্রয়োগ করতে হবে।
3. **Solution:** ক্লাস, দায়িত্ব ও তাদের সহযোগিতার বিমূর্ত বর্ণনা।
4. **Consequences:** প্রয়োগের সুবিধা ও মূল্য (trade-off)।

6.2 Why Learn Design Patterns

- **পরীক্ষিত সমাধান:** অভিজ্ঞ ডিজাইনারদের অভিজ্ঞতা; নতুন করে চাকা আবিষ্কার ও ভুল করতে হয় না।
- **সাধারণ শব্দভাণ্ডার:** “এখানে Observer ব্যবহার করো” — দুই শব্দে পুরো ডিজাইন বোঝানো যায়।
- **ভালো মানের ডিজাইন:** loose coupling, high cohesion, reusability, extensibility।
- **দ্রুত উন্নয়ন ও সহজ রক্ষণাবেক্ষণ:** পরিচিত কাঠামো সহজে চেনা ও বদলানো যায়।
- **Framework বোঝা:** Java I/O-র wrapper গুলো Decorator, Swing-এর listener Observer, Runtime.getRuntime() Singleton।
- **OO নীতি বাস্তবে শেখা:** প্রতিটি pattern encapsulation, polymorphism ও composition-এর ব্যবহার দেখায়।

6.3 Two GoF Principles of Reusable Design

১. Program to an interface, not an implementation

ভেরিয়েবল ও parameter-এর টাইপ হিসেবে interface বা abstract class ব্যবহার করো, concrete ক্লাস নয়। তাহলে যেকোনো implementation বদলে বসানো যায়।

```
1 List<String> names = new ArrayList<>(); // টাইপ List (interface), তাই পরে LinkedList বসানো যাবে
2 Notification n = NotificationFactory.create("sms"); // client শুধু interface চেনে
3 n.send("01712345678", "Complaint received");
```

২. Favour object composition over class inheritance

আচরণ তৈরি করতে ক্লাস extend না করে অন্য object ধারণ করো। Composition runtime-এ বদলানো যায়, ক্লাস ছোট থাকে, আর গভীর ভঙ্গুর hierarchy তৈরি হয় না। যেমন SmartPhone ক্যামেরা ক্লাস extend না করে একটি ICamera ধারণ করে।

6.4 Categories of Design Patterns

Category	উদ্দেশ্য	Patterns (মোট = notes-এ আছে)
Creational	object কীভাবে তৈরি হবে তা নিয়ন্ত্রণ; তৈরির logic লুকিয়ে client-কে concrete ক্লাস থেকে মুক্ত রাখা	Factory, Abstract Factory, Singleton, Builder, Prototype
Structural	ক্লাস ও object কীভাবে জুড়ে বড় কাঠামো তৈরি করবে, নমনীয়তা বজায় রেখে	Adapter, Bridge, Filter, Composite, Decorator, Facade, Flyweight, Proxy
Behavioural	object-গুলো কীভাবে যোগাযোগ করবে ও দায়িত্ব ভাগ করবে	Command, Mediator, Observer, Chain of Responsibility, Iterator, State, Strategy, Template Method, Visitor, Memento, Interpreter

Creational pattern কেন গুরুত্বপূর্ণ: কোডজুড়ে ছড়ানো `new ConcreteClass()` coupling বাড়ায়। Creational pattern কোন object, কয়টি, কীভাবে তৈরি হবে — সেই সিদ্ধান্ত এক জায়গায় রাখে; ফলে instance সংখ্যা নিয়ন্ত্রণ, জটিল object নিরাপদে তৈরি এবং নতুন product সহজে যোগ করা যায়।

Structural pattern-এর অবদান: অসামঞ্জস্যপূর্ণ অংশ জোড়া লাগায় (Adapter), জটিল subsystem সহজ করে (Facade), part-whole tree বানায় (Composite), abstraction ও implementation আলাদা করে (Bridge), subclass ছাড়াই দায়িত্ব যোগ করে (Decorator)। (Q&A: Q33, Q34)

পরীক্ষায় pattern-এর উত্তর লেখার ফর্মুলা

(১) Pattern-এর নাম ও category। (২) এক লাইনে intent। (৩) সিনারিওর ক্লাসগুলোকে pattern-এর ভূমিকার সাথে মেলাও। (৪) UML class diagram। (৫) Java কোড: interface, ১-২টি concrete ক্লাস, ছোট main। (৬) সিনারিও অনুযায়ী ২-৩টি সুবিধা।

Creational Patterns

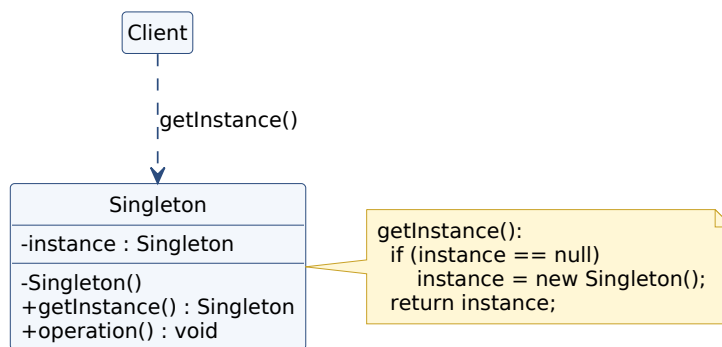
7.1 Singleton Pattern

Intent ও সমস্যা

একটি ক্লাসের শুধু একটিই instance থাকবে এবং সেটি পাওয়ার জন্য একটি **global access point** থাকবে। যেমন: logger, configuration manager, database connection pool, SMS gateway, printer spooler। একাধিক object হলে একই ফাইলে দুজন লিখে তথ্য নষ্ট করতে পারে, বা দুটি কনফিগারেশন ভিন্ন মান দিতে পারে, বা ব্যয়বহুল connection বারবার খোলা হতে পারে।

গঠন (Structure)

1. **private constructor:** বাইরে থেকে new করা যাবে না।
2. **private static field:** একমাত্র object-টি এখানে থাকবে।
3. **public static getInstance():** object না থাকলে তৈরি করে, থাকলে সেটাই ফেরত দেয়।



Singleton-এর সাধারণ গঠন

Notes-এর উদাহরণ: Eager initialization

```

1 public class SingleObject {
2     private static SingleObject instance = new SingleObject(); // ক্লাস লোড হওয়ার সময়েই একমাত্র
    object তৈরি
3
4     private SingleObject() { } // private constructor: বাইরে new করা নিষেধ
5
6     public static SingleObject getInstance() {
7         return instance; // সবসময় একই object ফেরত
8     }
9     public void showMessage() { System.out.println("Hello World!"); }
10 }
11
12 public class SingletonPatternDemo {
13     public static void main(String[] args) {
14         // SingleObject obj = new SingleObject(); // compile error: constructor private
15         SingleObject object = SingleObject.getInstance(); // একমাত্র বৈধ উপায়
16         object.showMessage();
    }
}

```

```

17     }
18 }

```

Lazy ও thread-safe সংস্করণ

```

1 public class Logger {
2     private static volatile Logger instance;    // volatile: অর্ধেক তৈরি object অন্য thread দেখবে না
3
4     private Logger() { }
5
6     public static Logger getInstance() {
7         if (instance == null) {                // প্রথম যাচাই: lock ছাড়া, দ্রুত
8             synchronized (Logger.class) {    // একবারে একটিই thread ঢুকবে
9                 if (instance == null) {        // দ্বিতীয় যাচাই: lock-এর ভেতরে
10                     instance = new Logger();   // প্রথমবার ডাকলেই তৈরি (lazy)
11                 }
12             }
13         }
14         return instance;
15     }
16 }

```

সাধারণ lazy সংস্করণে (if (instance == null) instance = new ...) দুটি thread একসাথে null দেখলে দুটি object তৈরি হতে পারে — একে বলে **race condition**! সমাধান: synchronized মেথড, eager initialization, double-checked locking (উপরে), holder class বা enum singleton।

সুবিধা ও অসুবিধা

সুবিধা: একটিমাত্র instance নিশ্চিত, সব জায়গা থেকে সহজ প্রবেশ, resource সাশ্রয়, lazy হলে দরকার না পড়লে তৈরি হয় না।

অসুবিধা: global state তৈরি করে, dependency লুকিয়ে ফেলে, unit test-এ mock বসানো কঠিন, multithreading-এ সতর্কতা লাগে, অনেক সময় SRP ভঙ্গ করে (নিজের কাজ + instance নিয়ন্ত্রণ)।

পরীক্ষার প্রশ্নে “convert this class into Singleton” এলে

দেওয়া constructor-এ যদি parameter থাকে, getInstance-এও সেই parameter নিতে হবে। হাতে লেখা solve-এর new Database() ভুল, কারণ constructor-টি তিনটি argument নেয়। ২০২৬ Database, ২০২৩ Logger, ২০২৪ ConfigurationManager — পূর্ণ উত্তর Q&A: Q35, Q36, Q37।

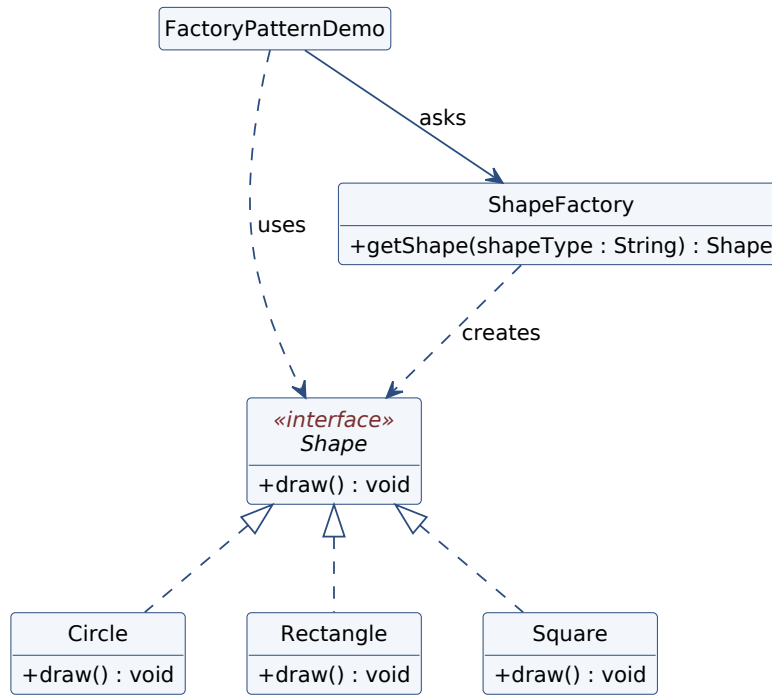
7.2 Factory Pattern

Intent ও সমস্যা

Client-কে তৈরির logic না দেখিয়ে object তৈরি করা এবং নতুন object-কে একটি **সাধারণ interface**-এর মাধ্যমে ব্যবহার করা। যখন কোডের নানা জায়গায় if (type.equals("circle")) new Circle() লেখা থাকে, তখন প্রতিটি client প্রতিটি concrete ক্লাসের উপর নির্ভরশীল হয় আর নতুন টাইপ এলে সব জায়গা বদলাতে হয়।

অংশগ্রহণকারী

Product interface (Shape), **Concrete products** (Circle, Rectangle, Square), **Factory** (ShapeFactory), **Client** (FactoryPatternDemo)।



Notes-এর Factory উদাহরণ

```

1 public interface Shape { void draw(); } // সব product-এর সাধারণ interface
2
3 public class Circle implements Shape {
4     public void draw() { System.out.println("Inside Circle::draw() method."); }
5 }
6 public class Rectangle implements Shape {
7     public void draw() { System.out.println("Inside Rectangle::draw() method."); }
8 }
9
10 public class ShapeFactory {
11     public Shape getShape(String shapeType) { // টাইপ দেখে সঠিক object তৈরি
12         if (shapeType == null) return null;
13         if (shapeType.equalsIgnoreCase("CIRCLE")) return new Circle();
14         if (shapeType.equalsIgnoreCase("RECTANGLE")) return new Rectangle();
15         if (shapeType.equalsIgnoreCase("SQUARE")) return new Square();
16         return null;
17     }
18 }
19
20 public class FactoryPatternDemo {
21     public static void main(String[] args) {
22         ShapeFactory shapeFactory = new ShapeFactory();
23         Shape shape1 = shapeFactory.getShape("CIRCLE"); // client জানে না কোন ক্লাস তৈরি হলো
24         shape1.draw(); // শুধু interface-এর মেথড ব্যবহার
25     }
26 }
  
```

সুবিধা: client concrete ক্লাস থেকে মুক্ত (loose coupling), তৈরির logic এক জায়গায়, নতুন product যোগ করা সহজ।
DUMS-এ SMS/Email/Push/WhatsApp notification — Q&A: Q38।

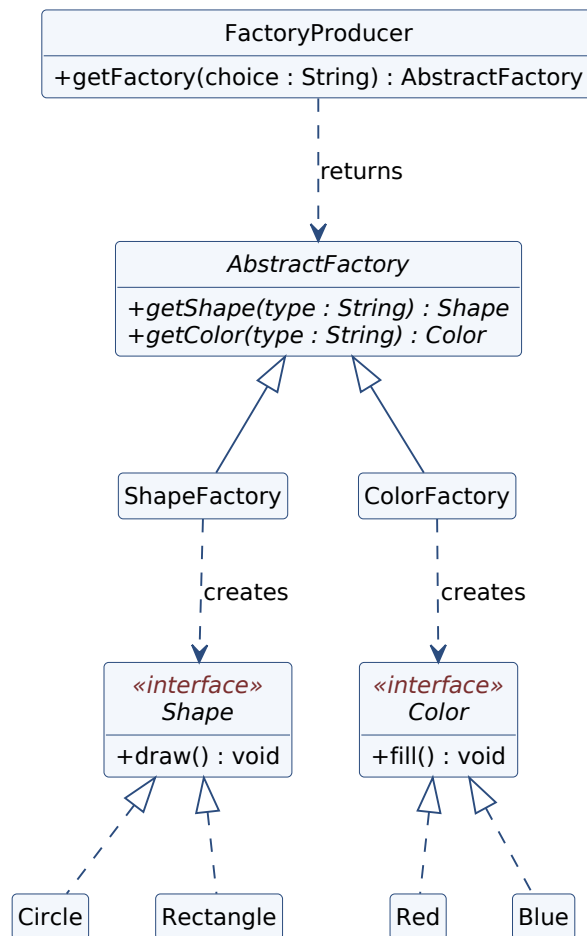
7.3 Abstract Factory Pattern

Intent

Concrete ক্লাস উল্লেখ না করে **পরস্পর সম্পর্কিত object-এর পরিবার (family)** তৈরি করার একটি interface দেওয়া। একে “factory of factories” বলা হয়: প্রথমে একটি factory বাছাই করা হয়, তারপর সেই factory পুরো পরিবারের object দেয়।

অংশগ্রহণকারী

AbstractFactory, **ConcreteFactory** (প্রতি পরিবারের জন্য একটি), **AbstractProduct**, **ConcreteProduct**, **FactoryProducer/Client**।



Notes-এর Shape/Color abstract factory

```

1 public abstract class AbstractFactory {
2     abstract Shape getShape(String shape); // পরিবারের এক ধরনের product
3     abstract Color getColor(String color); // পরিবারের আরেক ধরনের product
4 }
5
6 public class ShapeFactory extends AbstractFactory {
7     Shape getShape(String shapeType) {
8         if (shapeType.equalsIgnoreCase("CIRCLE")) return new Circle();
9         return null;
10    }
11    Color getColor(String color) { return null; } // এই factory রং বানায় না
  
```

```

12 }
13
14 public class FactoryProducer {
15     public static AbstractFactory getFactory(String choice) { // কোন factory লাগবে তা বাছাই
16         if (choice.equalsIgnoreCase("SHAPE")) return new ShapeFactory();
17         if (choice.equalsIgnoreCase("COLOR")) return new ColorFactory();
18         return null;
19     }
20 }
21
22 // client
23 AbstractFactory shapeFactory = FactoryProducer.getFactory("SHAPE");
24 Shape shape = shapeFactory.getShape("CIRCLE"); // factory থেকে product
25 shape.draw();

```

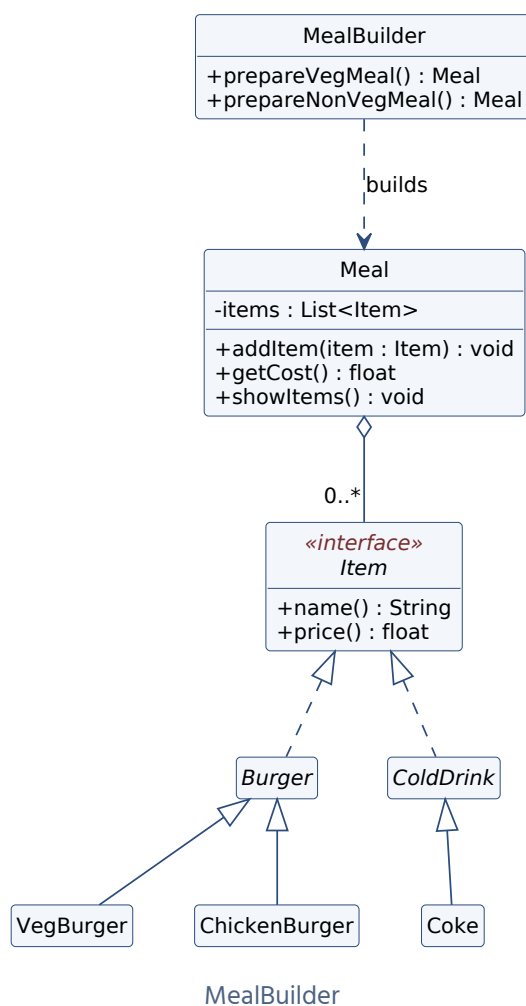
Factory বনাম Abstract Factory: Factory একটি product টাইপের কোন subclass তৈরি হবে তা ঠিক করে; Abstract Factory একাধিক সম্পর্কিত product টাইপের পুরো পরিবার তৈরি করে এবং নিশ্চিত করে যে একই পরিবারের object একসাথে ব্যবহার হবে। AAS উদাহরণ (AgroTech sensor + valve): Q&A: Q39।

7.4 Builder Pattern

Intent ও সমস্যা

একটি জটিল object-এর তৈরি প্রক্রিয়াকে তার উপস্থাপনা থেকে আলাদা করা, যাতে ধাপে ধাপে তৈরি করা যায় এবং একই প্রক্রিয়ায় ভিন্ন ভিন্ন উপস্থাপনা পাওয়া যায়। যখন object-এর অনেক ট্রেস্টিক অংশ থাকে, তখন অসংখ্য overloaded constructor (telescoping constructor) তৈরি হয় যা পড়া ও ব্যবহার করা কঠিন।

Notes-এর উদাহরণ: Meal Builder



```

1 public class Meal {
2     private List<Item> items = new ArrayList<Item>();
3     public void addItem(Item item) { items.add(item); } // ধাপে ধাপে অংশ যোগ
4     public float getCost() {
5         float cost = 0.0f;
6         for (Item item : items) cost += item.price(); // সব অংশের দাম যোগ
7         return cost;
8     }
9 }
10
11 public class MealBuilder {
12     public Meal prepareVegMeal() { // একই প্রক্রিয়া, ভিন্ন উপস্থাপনা
13         Meal meal = new Meal();
14         meal.addItem(new VegBurger());
15         meal.addItem(new Coke());
16         return meal;
17     }
18     public Meal prepareNonVegMeal() {
19         Meal meal = new Meal();
20         meal.addItem(new ChickenBurger());
21         meal.addItem(new Pepsi());

```



```
22     return meal;  
23 }  
24 }
```

আধুনিক রূপ: static nested Builder

বাস্যতামূলক তথ্য Builder-এর constructor-এ, প্রতিটি ঐচ্ছিক তথ্যের জন্য নামযুক্ত chainable মেথড (return this), এবং build()-এ যাচাই করে মূল ক্লাসের private constructor ডাকা হয়:

```
1 HousingApplication app = new HousingApplication.Builder("Rahima Begum", "1990123456")  
2     .familySize(5) // প্রতিটি মান নাম দিয়ে --- ভুল হওয়ার সুযোগ কম  
3     .preferredArea("Mirpur")  
4     .build(); // এখানে যাচাই, তারপর object তৈরি
```

সুবিধা: পাঠযোগ্য কোড, ঐচ্ছিক মানের default, একবারে যাচাই, immutable object, নতুন field যোগ করলে constructor-এর সংখ্যা বাড়ে না। পূর্ণ কোড: Q&A: Q40।

Structural Patterns

8.1 Adapter Pattern

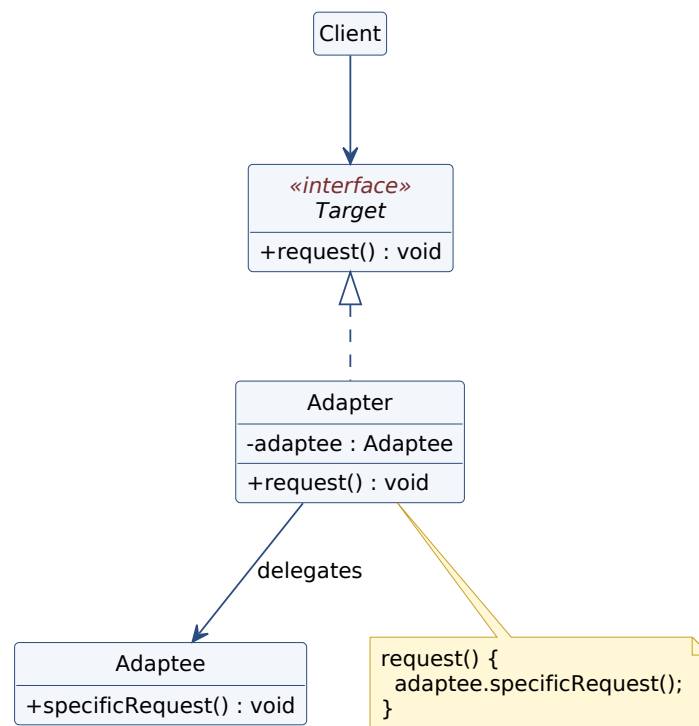
Intent ও সমস্যা

অসামঞ্জস্যপূর্ণ interface-এর দুটি ক্লাসকে একসাথে কাজ করানো। Client একটি interface (Target) আশা করে, কিন্তু বিদ্যমান ক্লাসটি (Adaptee, প্রায়ই third-party লাইব্রেরি) ভিন্ন interface দেয়, এবং কোনোটিই বদলানো সম্ভব নয়। Adapter ক্লাস Target implement করে এবং প্রতিটি কলকে Adaptee-র কলে রূপান্তর করে। একে **Wrapper**-ও বলে।

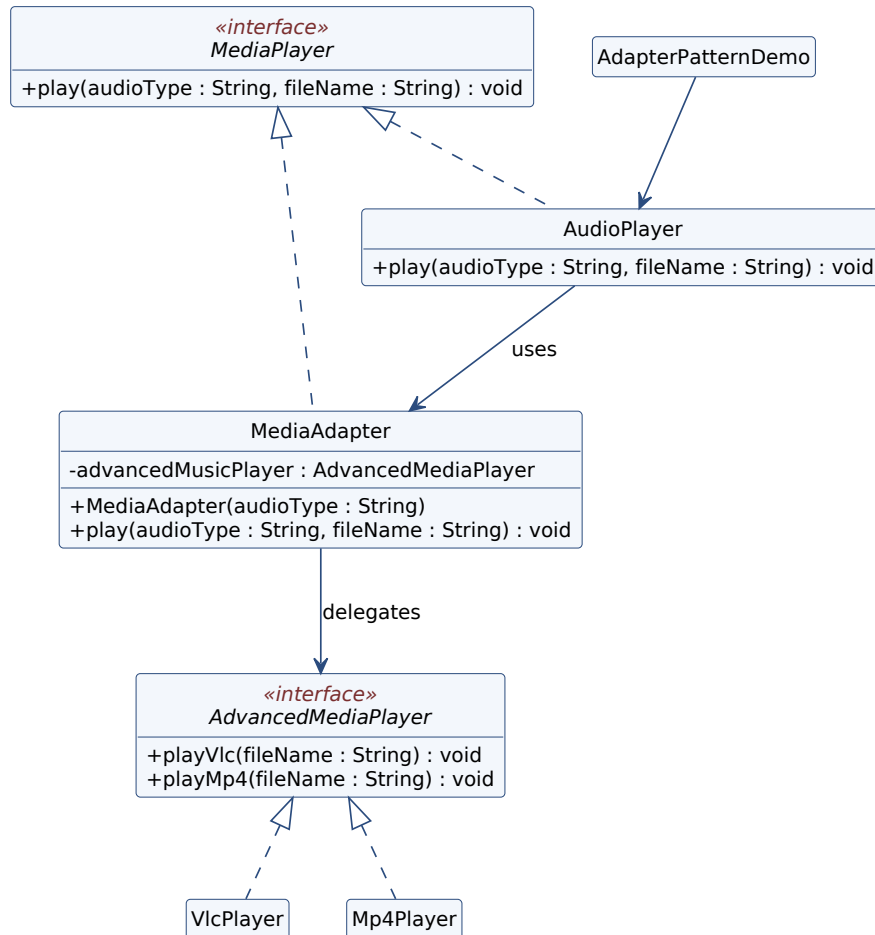
বাস্তব উদাহরণ: ভ্রমণের plug adapter — তিন-পিনের প্লাগ দুই-পিনের সকেটে লাগানো, ল্যাপটপ বা দেয়াল কিছুই না বদলে।
কার্ড রিডার — SD কার্ড USB পোর্টে ব্যবহার।

অংশগ্রহণকারী

Target (client যে interface চায়), **Adaptee** (বিদ্যমান অসামঞ্জস্যপূর্ণ ক্লাস), **Adapter** (Target implement করে, Adaptee ধারণ করে, কল রূপান্তর করে), **Client**।



Adapter-এর সাধারণ গঠন



Notes-এর media player adapter

```

1 public class MediaPlayerAdapter implements MediaPlayer { // Target implement করছে
2     AdvancedMediaPlayer advancedMusicPlayer; // Adaptee ধারণ করছে (composition)
3
4     public MediaPlayerAdapter(String audioType) {
5         if (audioType.equalsIgnoreCase("vlc")) advancedMusicPlayer = new VlcPlayer();
6         else if (audioType.equalsIgnoreCase("mp4")) advancedMusicPlayer = new Mp4Player();
7     }
8
9     @Override
10    public void play(String audioType, String fileName) { // client-এর পরিচিত মেথড
11        if (audioType.equalsIgnoreCase("vlc"))
12            advancedMusicPlayer.playVlc(fileName); // Adaptee-র ভিন্ন নামের মেথডে রূপান্তর
13        else if (audioType.equalsIgnoreCase("mp4"))
14            advancedMusicPlayer.playMp4(fileName);
15    }
16 }
17
18 public class AudioPlayer implements MediaPlayer {
19     MediaPlayerAdapter mediaAdapter;
20     @Override
21     public void play(String audioType, String fileName) {
22         if (audioType.equalsIgnoreCase("mp3")) {
23             System.out.println("Playing mp3 file. Name: " + fileName); // নিজের পুরনো ক্ষমতা

```

```

24     } else if (audioType.equalsIgnoreCase("vlc") || audioType.equalsIgnoreCase("mp4")) {
25         mediaAdapter = new MediaAdapter(audioType);    // নতুন ফরম্যাট adapter-এর মাধ্যমে
26         mediaAdapter.play(audioType, fileName);
27     } else {
28         System.out.println("Invalid media. " + audioType + " format not supported");
29     }
30 }
31 }

```

Object adapter বনাম Class adapter

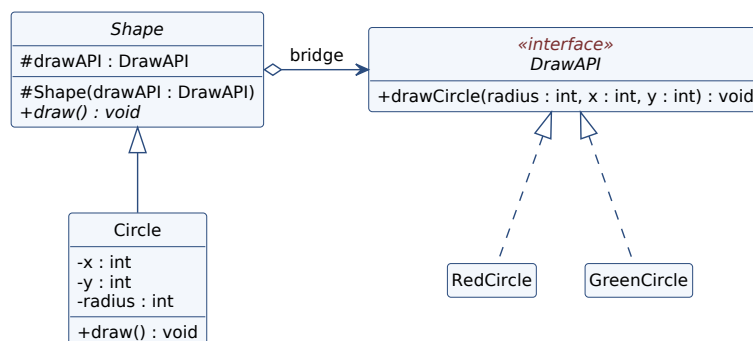
Object adapter Adaptee-কে ধারণ করে (composition) — Adaptee-র যেকোনো subclass-এর সাথে কাজ করে, final ক্লাসেও চলে, তাই সাধারণত এটাই পছন্দনীয়। **Class adapter** Adaptee-কে extend করে ও Target implement করে (inheritance) — Adaptee-র আচরণ override করা সহজ, কিন্তু একটি নির্দিষ্ট ক্লাসে আবদ্ধ এবং Java-তে একটিমাত্র superclass হওয়ায় সীমিত।

সুবিধা: পুরনো কোড ও লাইব্রেরি না বদলে পুনর্ব্যবহার (OCP), রূপান্তরের logic এক ক্লাসে (SRP), client শুধু Target চেনে। DUMS ও AAS উদাহরণ (IoT soil sensor — CT-2, mobile wallet): Q&A: Q41, Q42, Q43।

8.2 Bridge Pattern

Intent ও সমস্যা

Abstraction-কে তার implementation থেকে আলাদা করা, যাতে দুটো স্বাধীনভাবে পরিবর্তিত হতে পারে। শুধু inheritance ব্যবহার করলে দুটি মাত্রা (যেমন আকৃতি $\times$ রং, বা alert-এর ধরন $\times$ চ্যানেল) মিলে $m \times n$ সংখ্যক ক্লাস তৈরি হয় (class explosion)। Bridge-এ abstraction একটি implementor interface-এর রেফারেন্স রাখে — এই রেফারেন্সটাই “bridge” — ফলে $m + n$ ক্লাসেই কাজ হয়।



Notes-এর Bridge উদাহরণ

```

1 public interface DrawAPI {                                // Implementor
2     public void drawCircle(int radius, int x, int y);
3 }
4 public class RedCircle implements DrawAPI {                // Concrete implementor
5     public void drawCircle(int radius, int x, int y) {
6         System.out.println("Drawing Circle[ color: red, radius: " + radius + ", x: " + x + "
7         , " + y + " ]");
8     }
9 }

```

```

10 public abstract class Shape { // Abstraction
11     protected DrawAPI drawAPI; // এটাই bridge: implementation-এর রেফারেন্স
12     protected Shape(DrawAPI drawAPI) { this.drawAPI = drawAPI; }
13     public abstract void draw();
14 }
15 public class Circle extends Shape { // Refined abstraction
16     private int x, y, radius;
17     public Circle(int x, int y, int radius, DrawAPI drawAPI) {
18         super(drawAPI);
19         this.x = x; this.y = y; this.radius = radius;
20     }
21     public void draw() { drawAPI.drawCircle(radius, x, y); } // আসল আঁকার কাজ implementor করে
22 }
23
24 // client
25 Shape redCircle = new Circle(100, 100, 10, new RedCircle()); // রং runtime-এ ঠিক হলো
26 Shape greenCircle = new Circle(100, 100, 10, new GreenCircle());
27 redCircle.draw();

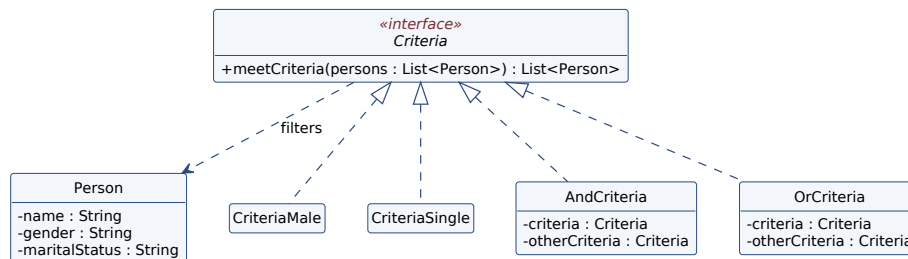
```

Bridge বনাম Adapter: Adapter ডিজাইনের পরে বিদ্যমান অসামঞ্জস্যপূর্ণ ক্লাস জোড়া লাগায়; Bridge ডিজাইনের শুরুতেই পরিকল্পনা করা হয় যাতে দুই দিক আলাদাভাবে বাড়ে। DUMS alert উদাহরণ: Q&A: Q46।

8.3 Filter (Criteria) Pattern

Intent

একটি object সংগ্রহকে **ভিন্ন ভিন্ন শর্ত (criteria)** দিয়ে ছাঁকা এবং logical operation (AND, OR) দিয়ে শর্তগুলোকে **de-coupled** উপায়ে জোড়া লাগানো। প্রতিটি শর্ত একটি আলাদা ক্লাস যা একটি সাধারণ interface implement করে।



Notes-এর Person filter

```

1 public interface Criteria {
2     public List<Person> meetCriteria(List<Person> persons); // তালিকা নিয়ে ছাঁকা তালিকা ফেরত
3 }
4
5 public class CriteriaMale implements Criteria {
6     public List<Person> meetCriteria(List<Person> persons) {
7         List<Person> malePersons = new ArrayList<Person>();
8         for (Person person : persons) {
9             if (person.getGender().equalsIgnoreCase("MALE")) // শুধু একটি শর্ত পরীক্ষা
10                 malePersons.add(person);
11         }
12         return malePersons;
13     }
14 }

```

```

13     }
14 }
15
16 public class AndCriteria implements Criteria {
17     private Criteria criteria;
18     private Criteria otherCriteria;
19     public AndCriteria(Criteria criteria, Criteria otherCriteria) {
20         this.criteria = criteria;
21         this.otherCriteria = otherCriteria;
22     }
23     public List<Person> meetCriteria(List<Person> persons) {
24         List<Person> first = criteria.meetCriteria(persons); // প্রথম শর্তে ছাঁকা
25         return otherCriteria.meetCriteria(first);           // সেই ফলাফলে দ্বিতীয় শর্ত
26     }
27 }

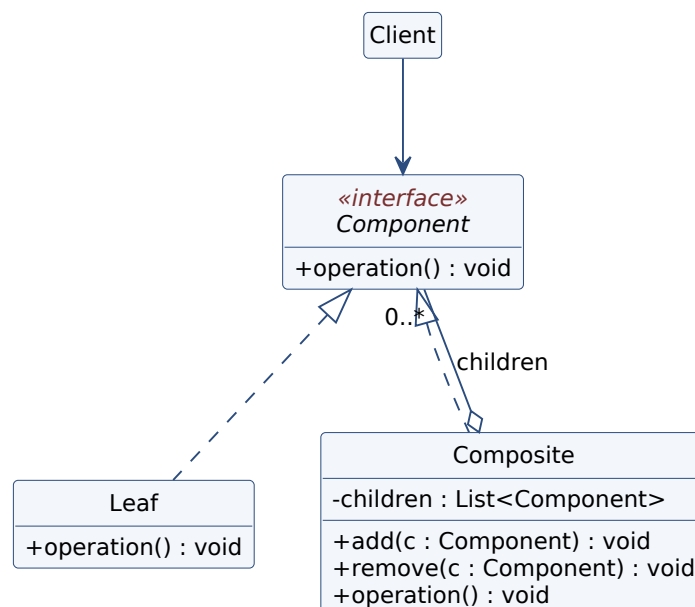
```

OrCriteria দুটি শর্তের ফলাফল একত্র (union) করে, পুনরাবৃত্তি বাদ দিয়ে। **সুবিধা:** প্রতিটি নিয়ম ছোট, পুনর্ব্যবহারযোগ্য, পরীক্ষাযোগ্য; নতুন নিয়ম যোগ করতে পুরনো কোড বদলাতে হয় না; runtime-এ যেকোনো সমস্বয় তৈরি করা যায়। DUMS complaint filter: Q&A: Q47।

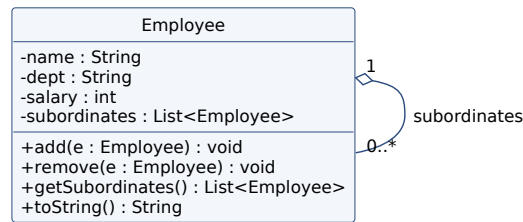
8.4 Composite Pattern

Intent

Object-গুলোকে **tree** কাঠামোতে সাজিয়ে part-whole hierarchy প্রকাশ করা এবং client যেন একক object ও object-এর সমষ্টিতে একইভাবে ব্যবহার করতে পারে। অংশগ্রহণকারী: **Component** (সাধারণ interface), **Leaf** (সন্তানহীন), **Composite** (সন্তানদের তালিকা রাখে ও কাজ তাদের উপর চালায়), **Client**।



Composite-এর সাধারণ গঠন



Notes-এর Employee উদাহরণ

```

1 public class Employee {
2     private String name;
3     private String dept;
4     private int salary;
5     private List<Employee> subordinates;           // একই টাইপের সন্তান: tree তৈরি হয়
6
7     public Employee(String name, String dept, int sal) {
8         this.name = name; this.dept = dept; this.salary = sal;
9         subordinates = new ArrayList<Employee>();
10    }
11    public void add(Employee e) { subordinates.add(e); } // শাখা যোগ
12    public void remove(Employee e) { subordinates.remove(e); }
13    public List<Employee> getSubordinates() { return subordinates; }
14    public String toString() {
15        return "Employee :[ Name : " + name + ", dept : " + dept + ", salary : " + salary + "
16        ]";
17    }
18 }
19 // client: CEO -> Head -> Clerk
20 Employee CEO = new Employee("John", "CEO", 30000);
21 Employee headSales = new Employee("Robert", "Head Sales", 20000);
22 CEO.add(headSales);                               // composite-এর ভেতরে composite
23 headSales.add(new Employee("Richard", "Sales", 10000)); // leaf
24 for (Employee e : CEO.getSubordinates()) System.out.println(e); // একইভাবে সবাইকে ব্যবহার
  
```

সুবিধা: client-এর কোড সরল ও একরূপ; নতুন স্তর বা leaf সহজে যোগ; মোট হিসাব, প্রিন্ট, খোঁজার মতো কাজ স্বাভাবিকভাবেই recursive। DUMS City → Zone → Ward → Bin: Q&A: Q45।

8.5 Facade Pattern

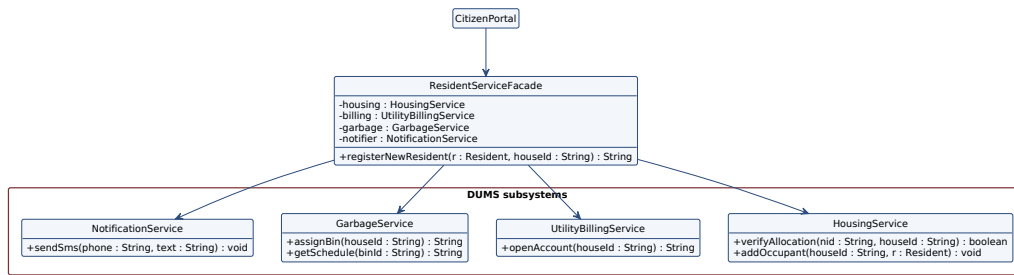
Intent

একটি subsystem-এর অনেকগুলো interface-এর জন্য একটি একীভূত, সরল interface দেওয়া। Facade নতুন কাজ যোগ করে না; বিদ্যমান subsystem ক্লাসগুলোকে সঠিক ক্রমে ডেকে জটিলতা লুকায়। (Notes-এ নেই, কিন্তু ২০২৬-এ পুরো একটি প্রশ্ন এসেছে।)

প্রয়োগের ধাপ

1. Subsystem-এর ক্লাস ও client-এর প্রয়োজনীয় উচ্চ-স্তরের কাজ চিহ্নিত করো।
2. একটি facade ক্লাস বানাও যা subsystem object-গুলোর রেফারেন্স রাখে।
3. প্রতিটি উচ্চ-স্তরের কাজের জন্য facade-এ একটি সরল মেথড দাও যা সঠিক ক্রমে subsystem ডাকে।

4. Client-কে facade ব্যবহার করাও; তবে প্রয়োজনে subsystem সরাসরি ব্যবহারের পথ খোলা রাখো।



DUMS resident registration facade

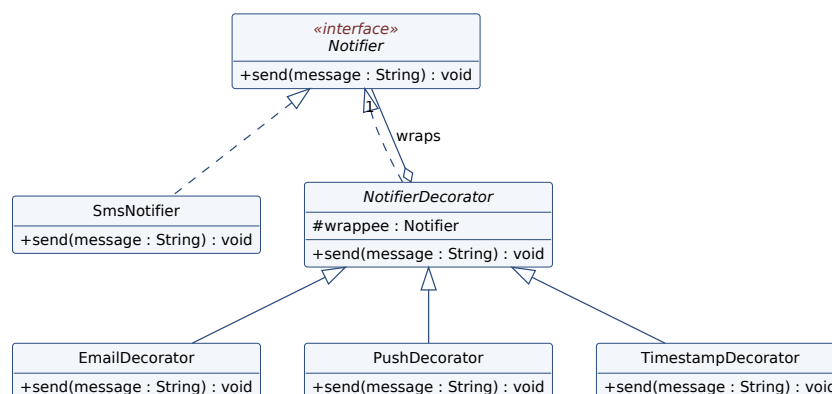
```

1 public class ResidentServiceFacade {
2     private final HousingService housing = new HousingService();
3     private final UtilityBillingService billing = new UtilityBillingService();
4     private final GarbageService garbage = new GarbageService();
5     private final NotificationService notifier = new NotificationService();
6
7     public String registerNewResident(Resident r, String houseId) { // client-এর জন্য একটিমাত্র
        মেথড
8         housing.addOccupant(houseId, r); // ধাপ ১: housing
9         String account = billing.openAccount(houseId); // ধাপ ২: billing
10        String bin = garbage.assignBin(houseId); // ধাপ ৩: garbage
11        notifier.sendSms(r.phone, "Welcome! Account " + account); // ধাপ ৪: notification
12        return account;
13    }
14 }
  
```

Facade যে সুবিধা দেয়: সরলতা, client ও subsystem-এর মধ্যে loose coupling, layering (প্রতিটি স্তরের একটি প্রবেশপথ), সঠিক ক্রম এক জায়গায় লেখা (duplication কম), প্রয়োজনে subsystem-এ সরাসরি প্রবেশের নমনীয়তা। Q&A: Q44।

8.6 Decorator Pattern (Extra)

Intent: একটি object-এ runtime-এ গতিশীলভাবে নতুন দায়িত্ব যোগ করা, subclass ছাড়াই। Decorator মূল object-এর মতো একই interface implement করে, ভেতরে মূল object ধারণ করে, কল তাকে পাঠায় এবং আগে বা পরে নিজের কাজ যোগ করে। একাধিক decorator যেকোনো ক্রমে স্তূপ করা যায়। Inheritance-এ প্রতিটি সমন্বয়ের জন্য আলাদা subclass লাগত। Java I/O: new BufferedReader(new InputStreamReader(System.in))। (নিম্ন অগ্রাধিকার; Q&A: Q48)



Notifier decorator

Behavioural Patterns and MVC

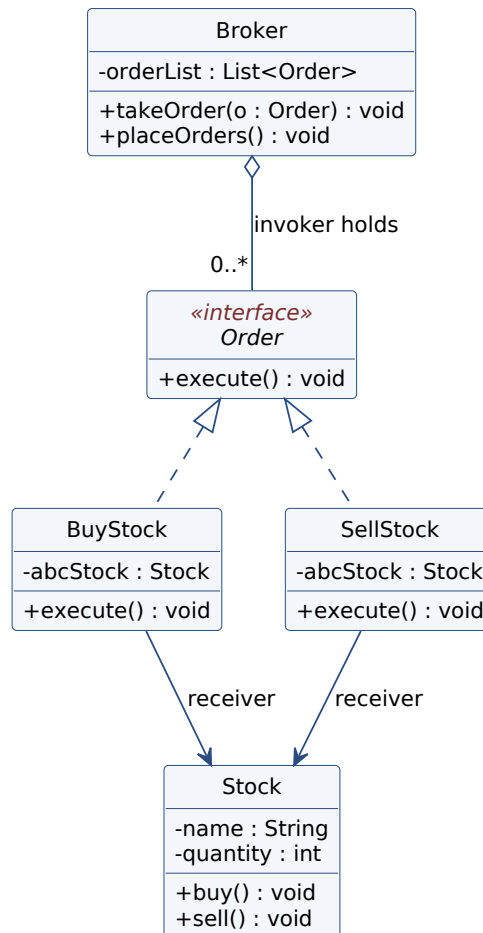
9.1 Command Pattern

Intent

একটি অনুরোধকে (request) object হিসেবে encapsulate করা, যাতে ভিন্ন ভিন্ন অনুরোধ দিয়ে client-কে parameterize করা যায়, অনুরোধ queue বা log করা যায় এবং undo সমর্থন করা যায়।

অংশগ্রহণকারী

ভূমিকা	দায়িত্ব	Notes-এর উদাহরণে
Command	execute() ঘোষণা করে	Order
Concrete Command	receiver ও action-কে বাঁধে	BuyStock, SellStock
Receiver	আসল কাজ জানে ও করে	Stock
Invoker	command জমা রাখে ও execute করায়	Broker
Client	command তৈরি করে invoker-কে দেয়	CommandPatternDemo



Notes-এর Command উদাহরণ

```

1 public interface Order { void execute(); }           // Command
2
3 public class Stock {                                // Receiver: আসল কাজ এখানে
4     private String name = "ABC";
5     private int quantity = 10;
6     public void buy() { System.out.println("Stock [ Name: " + name + ", Quantity: " +
7         quantity + " ] bought"); }
8     public void sell() { System.out.println("Stock [ Name: " + name + ", Quantity: " +
9         quantity + " ] sold"); }
10 }
11
12 public class BuyStock implements Order {             // Concrete command
13     private Stock abcStock;
14     public BuyStock(Stock abcStock) { this.abcStock = abcStock; }
15     public void execute() { abcStock.buy(); }         // receiver-কে দিয়ে কাজ করানো
16 }
17
18 public class Broker {                                // Invoker
19     private List<Order> orderList = new ArrayList<Order>();
20     public void takeOrder(Order order) { orderList.add(order); } // অনুরোধ জমা (queue)
21     public void placeOrders() {
22         for (Order order : orderList) order.execute(); // কী কাজ তা না জেনেই execute
23         orderList.clear();
24     }
25 }
26
27 // client
28 Stock abcStock = new Stock();
29 Broker broker = new Broker();
30 broker.takeOrder(new BuyStock(abcStock));
31 broker.takeOrder(new SellStock(abcStock));
32 broker.placeOrders();                                // পরে একসাথে সব চালানো

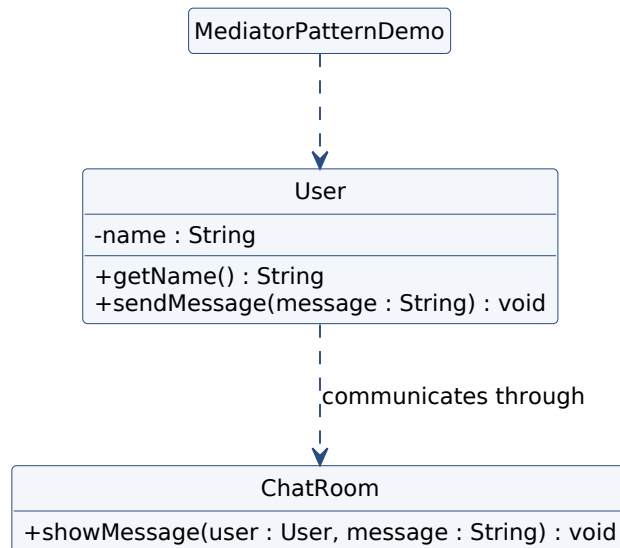
```

সুবিধা: invoker ও receiver সম্পূর্ণ আলাদা; নতুন command যোগ করতে invoker বদলাতে হয় না; queue, log, undo/redo, macro command সহজ। AAS farmer app (queue + undo): Q&A: Q55।

9.2 Mediator Pattern

Intent ও সমস্যা

একটি object তৈরি করা যা একদল object-এর পারস্পরিক যোগাযোগকে encapsulate করে। অনেক object সরাসরি একে অপরের সাথে কথা বললে many-to-many নির্ভরতার জাল তৈরি হয়; mediator সেটাকে one-to-many-তে নামিয়ে আনে এবং সমন্বয়ের নিয়ম এক জায়গায় রাখে।



Notes-এর ChatRoom mediator

```

1 public class ChatRoom {                                     // Mediator
2     public static void showMessage(User user, String message) { // সব বার্তা এখান দিয়ে যায়
3         System.out.println(new Date().toString() + " [" + user.getName() + "] : " + message)
4     }
5 }
6
7 public class User {                                         // Colleague
8     private String name;
9     public User(String name) { this.name = name; }
10    public String getName() { return name; }
11    public void sendMessage(String message) {
12        ChatRoom.showMessage(this, message);                // অন্য user-কে সরাসরি চেনে না
13    }
14 }
15
16 // client
17 User robert = new User("Robert");
18 User john = new User("John");
19 robert.sendMessage("Hi! John!");
20 john.sendMessage("Hello! Robert!");
  
```

Mediator বনাম Observer: Observer অজানা সংখ্যক subscriber-কে পরিবর্তনের খবর সম্প্রচার করে; Mediator পরিচিত একদল colleague-এর মধ্যে কে কী করবে তার নিয়ম সমন্বয় করে। DUMS traffic intersection: Q&A: Q56।

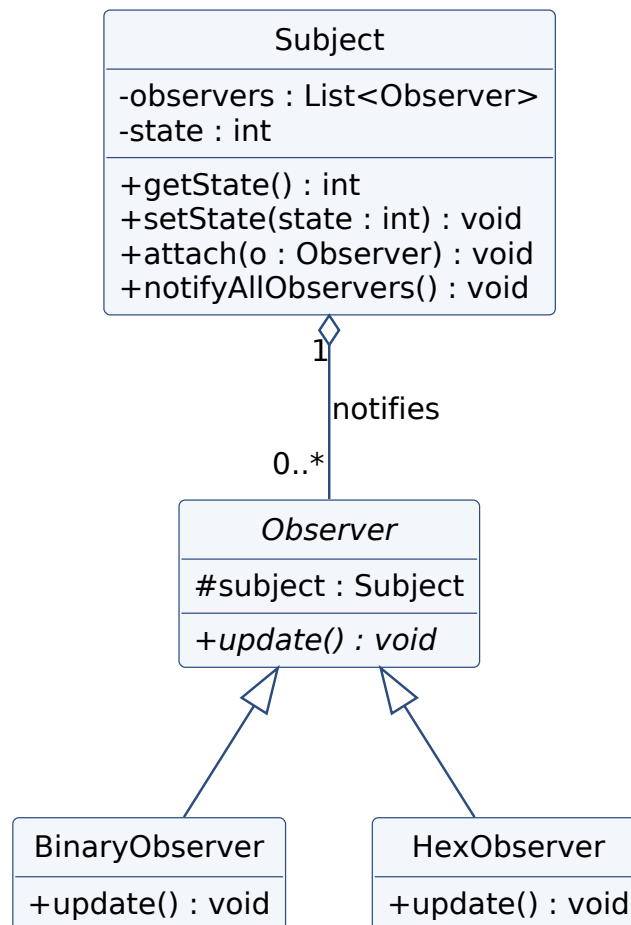
9.3 Observer Pattern

Intent

Object-গুলোর মধ্যে **one-to-many নির্ভরতা** সংজ্ঞায়িত করা, যাতে একটি object (**subject**) অবস্থা বদলালে তার সব নির্ভরশীল (**observer**) স্বয়ংক্রিয়ভাবে খবর পায় ও হালনাগাদ হয়। একে Publish-Subscribe-ও বলে।

মূল নীতি (২০২১-এর প্রশ্ন)

- **Subject ও observer-এর মধ্যে loose coupling:** subject শুধু জানে observer-দের একটি update() মেথড আছে; তারা কোন ক্লাস বা খবর দিয়ে কী করে তা জানে না।
- **Poll নয়, push:** observer বারবার “কিছু বদলেছে?” জিজ্ঞেস করে না; subject নিজেই জানায় (Hollywood principle: “don’t call us, we’ll call you”)।
- **গতিশীল subscription:** runtime-এ observer যোগ/বাদ দেওয়া যায়; নতুন observer টাইপ যোগ করতে subject বদলাতে হয় না (OCP)।



Notes-এর Observer উদাহরণ

```

1 public class Subject {
2     private List<Observer> observers = new ArrayList<Observer>(); // subscriber তালিকা
3     private int state;
4
5     public int getState() { return state; }
6     public void setState(int state) {
7         this.state = state;
8         notifyAllObservers(); // অবস্থা বদলালেই সবাইকে জানানো
9     }
10    public void attach(Observer observer) { observers.add(observer); }
11    public void notifyAllObservers() {
12        for (Observer observer : observers) observer.update(); // প্রত্যেককে update
13    }
  
```

```

14 }
15
16 public abstract class Observer {
17     protected Subject subject;
18     public abstract void update();           // খবর পেলে কী করবে তা subclass ঠিক করবে
19 }
20
21 public class BinaryObserver extends Observer {
22     public BinaryObserver(Subject subject) {
23         this.subject = subject;
24         this.subject.attach(this);         // নিজেকে subject-এর তালিকায় যুক্ত করা
25     }
26     @Override
27     public void update() {
28         System.out.println("Binary String: " + Integer.toBinaryString(subject.getState()));
29     }
30 }
31
32 // client
33 Subject subject = new Subject();
34 new HexaObserver(subject);
35 new BinaryObserver(subject);
36 subject.setState(15);                     // দুই observer-ই স্বয়ংক্রিয়ভাবে প্রিন্ট করবে

```

Push বনাম Pull: push মডেলে subject ডেটাসহ update(data) পাঠায়; pull মডেলে শুধু সংকেত দেয়, observer নিজে subject.getState() ডেকে ডেটা নেয় (উপরের উদাহরণ)।

সুবিধা: loose coupling, সম্প্রচার যোগাযোগ, নতুন observer সহজে যোগ। **অসুবিধা:** observer-দের notification-এর ক্রম নিশ্চিত নয়, অনেক observer হলে ধীর হতে পারে, unsubscribe না করলে memory leak।

পরীক্ষার সিনারিও: Facebook follower (২০২৪), DUMS complaint status (CT-2), AAS weather station — Q&A: Q49, Q50, Q51।

9.4 Chain of Responsibility Pattern

সিলেবাসে নেই মনে করো না

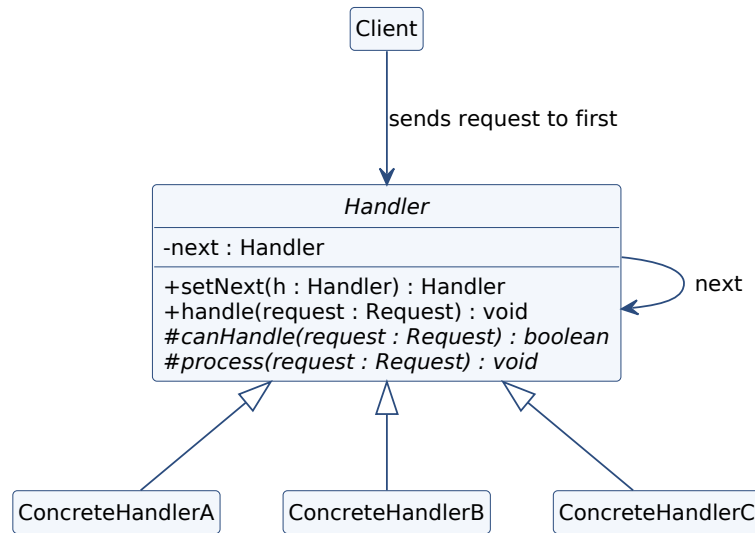
হাতে লেখা solve-এ বলা হয়েছে এটা সিলেবাসের বাইরে, কিন্তু ২০২৩ ফাইনালে (ব্যাংক চেক) ও CT-2-তে (Teacher → Chairman → Dean → VC) এসেছে।

Intent

একটি অনুরোধের **প্রেরক ও গ্রাহকের মধ্যে coupling এড়ানো**, একাধিক object-কে অনুরোধটি সামলানোর সুযোগ দিয়ে। Handler-গুলো একটি **শৃঙ্খলে (chain)** যুক্ত থাকে এবং অনুরোধটি শৃঙ্খল ধরে এগোয় যতক্ষণ না কেউ সেটা সামলায় (বা প্রয়োজনীয় সবাই সামলায়)।

অংশগ্রহণকারী

Handler (abstract; next রেফারেন্স ও handle()), **ConcreteHandler** (নিজে পারলে সামলায়, না পারলে পরের জনকে পাঠায়), **Client** (শুধু প্রথম handler-কে অনুরোধ দেয়)।



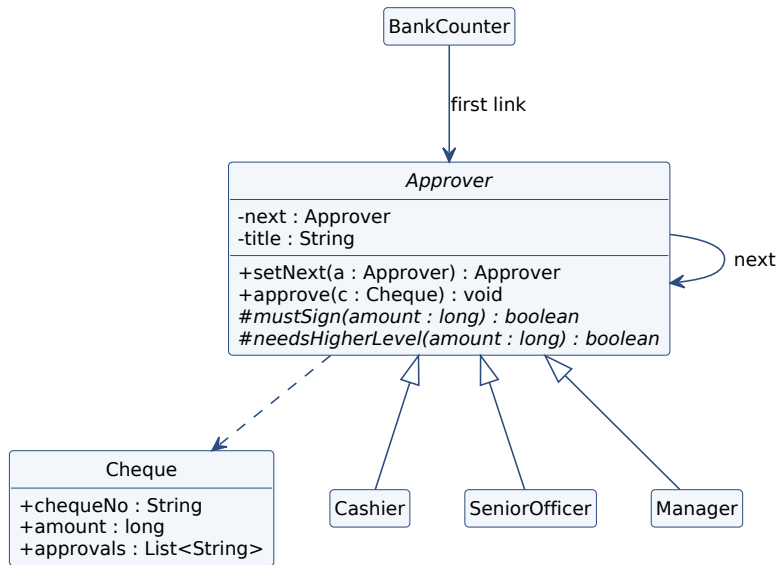
Chain of Responsibility-এর গঠন

```

1  abstract class Handler {
2      private Handler next; // শৃঙ্খলের পরের সদস্য
3
4      public Handler setNext(Handler next) {
5          this.next = next;
6          return next; // a.setNext(b).setNext(c) লেখা যায়
7      }
8
9      public void handle(int amount) {
10         if (canHandle(amount)) {
11             process(amount); // নিজের ক্ষমতায় থাকলে নিজেই করবে
12         } else if (next != null) {
13             next.handle(amount); // না পারলে পরের জনকে পাঠাবে
14         } else {
15             System.out.println("No one can handle " + amount); // শৃঙ্খলের শেষ
16         }
17     }
18     protected abstract boolean canHandle(int amount);
19     protected abstract void process(int amount);
20 }
21
22 class Teacher extends Handler {
23     protected boolean canHandle(int days) { return days <= 3; } // শিক্ষকের সীমা
24     protected void process(int days) { System.out.println("Teacher approved " + days + "
25         days"); }
26 }
27 // client
28 Handler chain = new Teacher();
29 chain.setNext(new Chairman()).setNext(new Dean()).setNext(new ViceChancellor());
30 chain.handle(10); // কে অনুমোদন দেবে client জানে না
  
```

দুই ধরনের chain: (১) প্রথম সক্ষম handler সামলালে থেমে যায় (university leave, complaint escalation); (২) প্রয়োজনীয় প্রত্যেকে নিজের অংশ করে পরের জনকে পাঠায় (ব্যংকের চেকে একাধিক স্বাক্ষর)।

সুবিধা: প্রেরক-গ্রাহক আলাদা, প্রতিটি handler-এর একটিই দায়িত্ব, runtime-এ handler যোগ/বাদ/ক্রম বদল। **অসুবিধা:** ভুল কনফিগারেশনে অনুরোধ অসামলানো থেকে যেতে পারে, লম্বা chain-এ debug কঠিন। পূর্ণ কোড: Q&A: Q52, Q53, Q54।

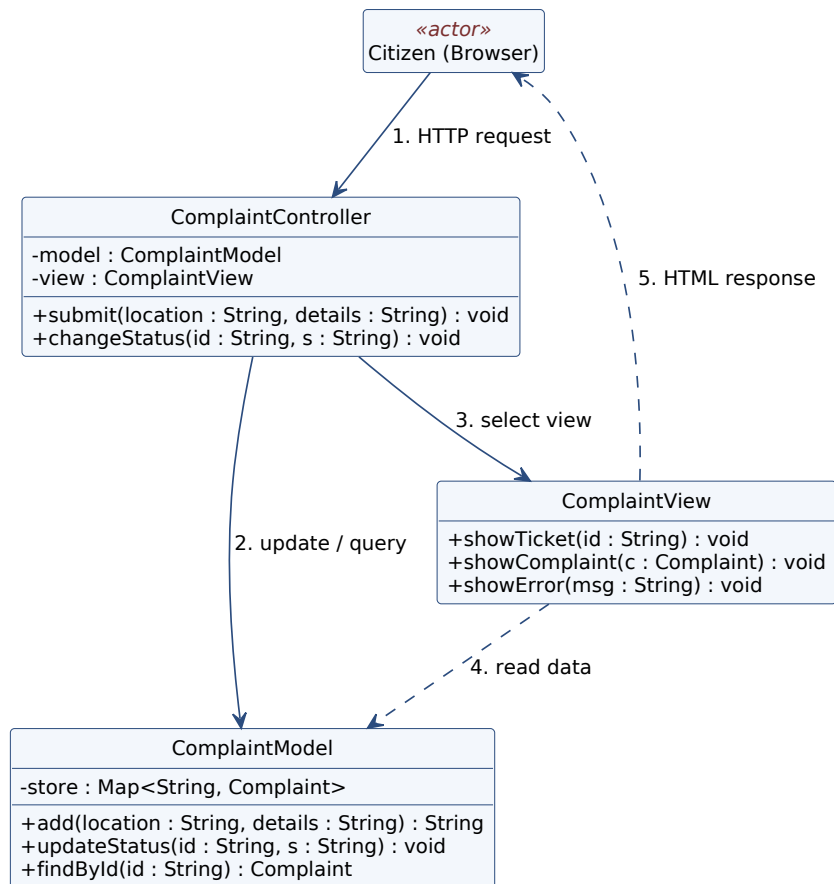


২০২৩-এর ব্যাংক চেক সমাধান

9.5 MVC (Model–View–Controller)

MVC একটি architectural/compound pattern যা ইন্টারঅ্যাক্টিভ অ্যাপ্লিকেশনকে তিন ভাগে ভাগ করে:

- **Model:** অ্যাপ্লিকেশনের ডেটা ও business rule; স্ক্রিন সম্পর্কে কিছু জানে না।
- **View:** Model-এর ডেটা ব্যবহারকারীকে দেখায় (HTML/JSON); কোনো business rule রাখে না।
- **Controller:** ব্যবহারকারীর অনুরোধ (HTTP request) নেয়, Model হালনাগাদ বা জিজ্ঞেস করে, এবং কোন View দেখাবে তা ঠিক করে।



DUMS complaint ওয়েব অ্যাপে MVC

Maintainable web application-এ MVC কেন (২০২১)

1. **Separation of concerns:** UI ডিজাইনার View, backend ডেভেলপার Model নিয়ে একে অপরকে বিরক্ত না করে কাজ করে।
2. **একই Model-এর একাধিক View:** ওয়েব পেজ, মোবাইল API, অ্যাডমিন ড্যাশবোর্ড।
3. **স্বাধীন পরিবর্তন:** HTML থেকে React-এ যাওয়া বা MySQL থেকে PostgreSQL-এ যাওয়া শুধু একটি স্তরে প্রভাব ফেলে।
4. **Testability:** ব্রাউজার ছাড়াই Model ও Controller-এর unit test।
5. **সমান্তরাল উন্নয়ন ও framework সহায়তা:** Spring MVC, Django, Laravel।
6. **Pattern-এর সমন্বয়:** View সাধারণত Model-কে observe করে (Observer), Controller ইনপুট সামলানোর strategy (Q&A: Q57)।

Quick Revision Tables

10.1 All Patterns at a Glance

Pattern	Category	এক লাইনে intent	DUMS/AAS উদাহরণ	Q&A
Singleton	Creational	একটিমাত্র instance ও global access	ConfigurationManager, Sms-Gateway	Q35–37
Factory	Creational	client-কে না জানিয়ে interface-এর object তৈরি	SMS/Email/Push notification	Q38
Abstract Factory	Creational	সম্পর্কিত object-এর পরিবার তৈরি	AgroTech sensor + valve	Q39
Builder	Creational	জটিল object ধাপে ধাপে তৈরি	HousingApplication	Q40
Adapter	Structural	অসামঞ্জস্যপূর্ণ interface জোড়া	IoT soil sensor, mobile wallet	Q41–43
Bridge	Structural	abstraction ও implementation আলাদা	Alert type × channel	Q46
Filter	Structural	শর্ত দিয়ে ছাঁকা ও জোড়া	Complaint by zone AND pending	Q47
Composite	Structural	tree; একক ও সমষ্টি একইভাবে	City→Zone→Ward→Bin	Q45
Facade	Structural	subsystem-এর সরল একক interface	Resident registration	Q44
Decorator	Structural	runtime-এ দায়িত্ব যোগ	SMS + Email + Push	Q48
Command	Behavioural	অনুরোধকে object বানানো; queue/undo	Farmer remote app	Q55
Mediator	Behavioural	যোগাযোগ কেন্দ্রীয় object-এর মাধ্যমে	Traffic intersection	Q56
Observer	Behavioural	one-to-many স্বয়ংক্রিয় notification	Complaint status, weather	Q49–51
Chain of Resp.	Behavioural	অনুরোধ শৃঙ্খল ধরে এগোয়	Complaint escalation, approval	Q52–54
MVC	Architectural	Model, View, Controller আলাদা	Complaint web app	Q57

10.2 Scenario Keywords → Pattern

প্রশ্নে এই শব্দ/পরিস্থিতি থাকলে	ভাবো
"single shared instance", "only one", "global access"	Singleton
"create without specifying concrete class", টাইপ দেখে object	Factory
"family of related products", vendor-ভিত্তিক সেট	Abstract Factory
"many optional parameters", "step by step"	Builder
"cannot modify the library", "incompatible interface"	Adapter
দুই দিকে স্বাধীন পরিবর্তন, class explosion	Bridge
"criteria", "AND/OR" দিয়ে ছাঁকা	Filter
"hierarchy", "tree", "part-whole", "treat uniformly"	Composite
"simplify complex subsystem", "single entry point"	Facade
"add features dynamically"	Decorator
"undo", "queue requests", "button actions"	Command
"many objects communicate", "central control"	Mediator
"notify all", "subscribe", "follow"	Observer
"levels of approval", "escalate", "pass to next"	Chain of Responsibility

10.3 Section A One-Page Summary

টপিক	মনে রাখার মূল কথা
Conceptual vs Technical	কী component (CRC, mock-up) বনাম কীভাবে বানানো (UML, API, অ্যালগরিদম)
CRC card	Class / Responsibilities / Collaborators; noun → class, verb → responsibility; role-play
Design vs Architecture	ভবনের ফ্লোর প্ল্যান বনাম শহরের নকশা
E/B/C objects	ডেটা / ইন্টারফেস / সমন্বয়
Abstraction	context অনুযায়ী দরকারি বৈশিষ্ট্যই রাখা
Encapsulation	bundling + private data + black box; getter/setter-এ যাচাই
Association / Aggregation / Composition	ব্যবহার / দুর্বল has-a (ফাঁপা হীরা) / শক্তিশালী part-of (ভরাট হীরা)
Generalization	is-a, DRY, polymorphism; LSP মেনে; ঐচ্ছিক ক্ষমতা interface-এ
Coupling	degree, ease, flexibility; flag = control coupling
Cohesion	একটি ক্লাস একটি উদ্দেশ্য; flag দিয়ে বাছাই = logical cohesion
SoC	concern অনুযায়ী আলাদা interface/ক্লাস, মূল ক্লাস সমন্বয় করে
Conceptual integrity	একজনের মস্তিষ্ক থেকে আসা মনে হবে; review, standards, architect, patterns
Code smells	Feature Envy, Switch, Message Chain, Refused Bequest, Comments ...
SRP / OCP / LoD	একটিই পরিবর্তনের কারণ / extension-এ খোলা, modification-এ বন্ধ / নিকটতম বন্ধুর সাথে কথা
Sequence diagram	lifeline, activation, sync/return, alt/opt/loop + guard
State diagram	state, event [guard] / action, entry/do/exit
Model checking	modelling → running → analysis (counterexample)